TRƯỜNG ĐẠI HỌC NAM CẦN THƠ
KHOA CÔNG NGHỆ THÔNG TIN

PHẠM TRUNG TÍNH - MSSV 226238

XÂY DỰNG HỆ THỐNG NHẬN DIỆN VẬT THỂ BẰNG AI

ĐỒ ÁN CƠ SỞ 2

Ngành: Công Nghệ Thông Tin

Mã ngành: 7480201

Tháng 03 - Năm 2026

TRƯỜNG ĐẠI HỌC NAM CẦN THƠ

KHOA CÔNG NGHỆ THÔNG TIN

PHẠM TRUNG TÍNH - MSSV 226238

XÂY DỰNG HỆ THỐNG NHẬN DIỆN VẬT THỂ BẰNG AI

ĐỒ ÁN CƠ SỞ 2

Ngành: Công Nghệ Thông Tin

Mã ngành: 7480201

CÁN BỘ HƯỚNG DẪN

TRẦN VĂN THIỆN

Tháng 03 - Năm 2026

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến quý thầy cô Khoa Công nghệ Thông tin - Trường Đại học Nam Cần Thơ đã tận tình giảng dạy, truyền đạt kiến thức và tạo điều kiện để em thực hiện đồ án cơ sở với đề tài "Xây dựng hệ thống nhận diện vật thể trong ảnh bằng trí tuệ nhân tạo".

Em xin chân thành cảm ơn quý thầy cô đã trang bị cho em nền tảng chuyên môn và phương pháp học tập nghiêm túc, giúp em có cơ sở để vận dụng vào quá trình nghiên cứu, xây dựng và hoàn thiện đề tài. Những góp ý chuyên môn, sự hướng dẫn tận tâm và động viên kịp thời của quý thầy cô là động lực quan trọng để em vượt qua khó khăn trong suốt quá trình thực hiện đồ án.

Em cũng kính mong tiếp tục nhận được ý kiến đóng góp từ quý thầy cô để em có thể hoàn thiện hơn nữa cả về kiến thức lẫn kỹ năng, đồng thời phát triển đề tài theo hướng thực tiễn và hiệu quả hơn trong thời gian tới.

Một lần nữa, em xin gửi lời cảm ơn chân thành đến quý thầy cô và nhà trường đã luôn đồng hành, hỗ trợ em trong suốt quá trình học tập. Kính chúc quý thầy cô thật nhiều sức khỏe, hạnh phúc và tiếp tục gặt hái nhiều thành công trong sự nghiệp trồng người.

Em xin chân thành cảm ơn!

Cần Thơ, Ngày ..... tháng ..... năm 2026

Sinh viên thực hiện

Phạm Trung Tính

LỜI CAM KẾT

Em xin cam kết đồ án này là kết quả nghiên cứu do bản thân thực hiện trên cơ sở kiến thức đã học tại trường, kết hợp với các tài liệu tham khảo liên quan đến đề tài. Nội dung và kết quả trình bày trong đồ án là trung thực, được thực hiện nghiêm túc trong quá trình học tập và chưa từng được sử dụng trong bất kỳ đồ án hoặc công trình nào trước đây. Nếu có sai phạm về tính trung thực học thuật, em xin hoàn toàn chịu trách nhiệm.

Cần Thơ, Ngày ..... tháng ..... năm 2026

Sinh viên thực hiện

Phạm Trung Tính

NHẬN XÉT, ĐÁNH GIÁ CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Cần Thơ, Ngày ..... tháng ..... năm 2026

Giảng viên hướng dẫn

Trần Văn Thiện

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI 1
1.1 Đặt vấn đề 1
1.2 Mục tiêu nghiên cứu 1
1.2 Mục tiêu nghiên cứu 2
1.2.1 Mục tiêu chung 2
1.3 Phạm vi nghiên cứu 4
1.3.1 Không gian nghiên cứu 4
1.3.2 Thời gian nghiên cứu 5
1.3.3 Phương pháp thực hiện 5
1.3.4 Đối tượng nghiên cứu 6
1.5 Ý nghĩa của đề tài 8
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 10
2.1 Tổng quan bài toán phát hiện và đếm vật thể 10
2.1.1 Tổng quan 10
2.1.2 Tổng quan mô hình YOLO và vai trò trong dự án YOLO 11
2.1.3 Nền tảng Streamlit và vai trò trong hệ thống Streamlit 11
2.2 Các tiêu chí đánh giá 12
2.3 Định hướng đề tài 13
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 15
3.1 Mô tả hệ thống 15
3.2 Kiến trúc tổng thể 16
3.3 Các thành phần chính 17
3.3.1 Ứng dụng suy luận và hiển thị (app.py) 17
3.3.2 Mô hình COCO (phát hiện đối tượng phổ biến) 18
3.3.3 Mô hình custom (dumbbell/flower/fruit/tree) 19
3.3.4 Cơ chế kết hợp hai mô hình (fusion + ưu tiên lớp) 20
3.3.5 Hậu xử lý và sinh kết quả cuối 21
CHƯƠNG 4: DỮ LIỆU VÀ TIỀN XỬ LÝ 22

4.1 Cấu trúc dataset và tổ chức tài nguyên 22
4.2 Normalize nhãn và xử lý alias 22
4.3 Dataset hoa chuyên sâu & augmentation 23
4.4 Kiểm tra chất lượng nhãn & monitoring 24
4.5 Roboflow & version tracking 25
CHƯƠNG 5: MÔ HÌNH HUẤN LUYỆN 27
5.2 Deploy & metadata governance 28
5.3 Chiến lược fine-tune theo lớp và rule context 28
5.4 Cấu hình huấn luyện và tài nguyên 29
5.5 Đánh giá training & benchmarking 30
5.6 Kiểm soát phiên bản triển khai và cơ chế rollback 31
CHƯƠNG 6: PIPELINE SUY LUẬN VÀ HẬU XỬ LÝ 32
6.1 Luồng suy luận song song 32
6.2 Hợp nhất detections và ưu tiên lớp 33
6.3 Rule hậu xử lý 35
6.4 Đếm số lượng và log kết quả 36
6.5 Khả năng fallback và sanity check 37
6.6 Giao diện hệ thống và minh họa kết quả 38
CHƯƠNG 7 – THỬ NGHIỆM KẾT QUẢ VÀ KẾT LUẬN 43
7.1 Thiết lập thử nghiệm 43
7.2 Kết quả đánh giá tổng thể 44
7.3 Kết quả đếm theo lớp 45
7.4 Phân tích lỗi và cải tiến 45
7.5 Kết luận và hướng phát triển 46
TÀI LIỆU THAM KHẢO 47

DANH MỤC HÌNH

Hình 3.1 Sơ đồ kiên trúc tổng thể 17
Hình 4.1 Sơ đồ pipeline dữ liệu và huấn luyện 26
Hình 5.1 Deploy metadata rollback 31
Hình 6.1 Sơ đồ luồng xử lý end-to-end 33
Hình 6.2 Sơ đồ hợp nhất detection và ưu tiên lớp 34
Hình 6.3 Sơ đồ rule hậu xử lý theo ngữ cảnh 36
Hình 6.4 Giao diện và tùy chọn model 38
Hình 6.5 Quy trình chọn và tải ảnh lên 38
Hình 6.6 Kết quả quét đúng 1 39
Hình 6.7 Kết quả quét đúng 2 39
Hình 6.8 Kết quả quét đúng 3 40
Hình 6.9 Kết quả quét đúng kết hợp hai model 40
Hình 6.10 Kết quả quét sai 1 41
Hình 6.11 Kết quả quét sai 2 42
Hình 6.12 Nhật ký hệ thống 42
Hình 7.1 Sơ đồ quy trình đánh giá 44

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Thuật ngữ tiếng Anh	Nghĩa tiếng Việt
1	AI	Artificial Intelligence	Trí tuệ nhân tạo
2	YOLO	You Only Look Once	Mô hình phát hiện vật thể một giai đoạn
3	COCO	Common Objects in Context	Bộ dữ liệu đối tượng phổ biến theo ngữ cảnh
4	NMS	Non-Maximum Suppression	Thuật toán loại bỏ dự đoán trùng lặp
5	IoU	Intersection over Union	Chỉ số giao nhau trên hợp nhất
6	mAP	mean Average Precision	Độ chính xác trung bình
7	mAP50	mean Average Precision at IoU = 0.50	mAP tại ngưỡng IoU bằng 0.50
8	mAP50–95	mean Average Precision at IoU 0.50–0.95	mAP trung bình trên dải IoU 0.50–0.95
9	MAE	Mean Absolute Error	Sai số tuyệt đối trung bình
10	TP	True Positive	Dự đoán đúng đối tượng
11	FP	False Positive	Dự đoán dương tính sai
12	FN	False Negative	Bỏ sót đối tượng thật
13	GPU	Graphics Processing Unit	Bộ xử lý đồ họa
14	CPU	Central Processing Unit	Bộ xử lý trung tâm
15	JSON	JavaScript Object Notation	Định dạng dữ liệu JSON
16	CSV	Comma-Separated Values	Định dạng dữ liệu phân tách bằng dấu phẩy
17	imgsz	Image size	Kích thước ảnh đầu vào mô hình
18	max_det	Maximum detections	Số lượng dự đoán tối đa trên mỗi ảnh

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Sự phát triển của chuyển đổi số làm gia tăng nhu cầu tự động hóa các tác vụ xử lý ảnh trong nhiều lĩnh vực như nông nghiệp thông minh, kiểm kê hàng hóa, giám sát sản xuất và phân tích hiện trường. Trong nhóm bài toán này, nhận diện và đếm vật thể từ ảnh tĩnh có ý nghĩa ứng dụng cao vì đầu ra không chỉ dừng ở mức phát hiện đối tượng mà còn cung cấp số lượng theo từng lớp để phục vụ thống kê và hỗ trợ ra quyết định.

Trong thực tế, dữ liệu ảnh thường không lý tưởng. Nhiều trường hợp xuất hiện ánh sáng không đồng đều, góc chụp thay đổi, đối tượng bị che khuất, kích thước vật thể nhỏ hoặc phân bố dày trong cùng một khung hình. Những yếu tố này dễ dẫn đến hai dạng sai số phổ biến là đếm thiếu và đếm thừa. Vì vậy, bài toán không thể giải quyết hiệu quả nếu chỉ dùng một mô hình nhận diện đơn lẻ mà cần một quy trình đầy đủ gồm chuẩn hóa dữ liệu, huấn luyện, suy luận, hợp nhất dự đoán, hậu xử lý theo ngữ cảnh và đánh giá định lượng.

Từ yêu cầu đó, đề tài lựa chọn hướng xây dựng hệ thống theo kiến trúc kết hợp giữa mô hình COCO và mô hình custom. Mô hình COCO giữ vai trò tăng khả năng tổng quát trong các bối cảnh dữ liệu phức tạp, trong khi mô hình custom tập trung vào các lớp mục tiêu gồm dumbbell, flower, fruit và tree. Sau bước suy luận song song, hệ thống áp dụng cơ chế hợp nhất kết quả cùng các quy tắc hậu xử lý để giảm trùng lặp bounding box, hạn chế nhầm lớp và ổn định số đếm cuối cùng.

Trong quá trình triển khai, lớp flower được xác định là lớp khó do mật độ xuất hiện cao và kích thước biến thiên lớn, dễ tạo dao động kết quả giữa các lần chạy. Vì lý do đó, đề tài bổ sung hướng xử lý chuyên sâu cho lớp này thông qua dữ liệu tăng cường và cơ chế specialist trong các tình huống cần thiết. Cách tiếp cận này giúp nâng chất lượng đầu ra theo hướng thực tiễn, đồng thời bảo đảm cân bằng giữa độ chính xác nhận diện và tính ổn định của bài toán đếm.

1.2 Mục tiêu nghiên cứu

Mục tiêu nghiên cứu của đề tài là xây dựng một hệ thống nhận diện và đếm vật thể trong ảnh theo hướng có thể vận hành ổn định trong điều kiện dữ liệu thực tế, thay vì chỉ đạt kết quả tốt trên tập kiểm thử lý tưởng. Hệ thống cần đồng thời đáp ứng hai yêu cầu cốt lõi gồm chất lượng phát hiện đối tượng theo lớp và độ tin cậy của kết quả đếm sau hậu xử lý. Trong bối cảnh triển khai thực tế, ảnh đầu vào thường biến động mạnh về ánh sáng, góc chụp, độ che khuất và mật độ vật thể, vì vậy mục tiêu nghiên cứu không dừng ở việc huấn luyện mô hình có mAP cao, mà mở rộng sang việc bảo đảm đầu ra có tính ổn định giữa nhiều bối cảnh khác nhau.

Trên phương diện kỹ thuật, đề tài đặt mục tiêu xây dựng cơ chế suy luận kết hợp giữa mô hình COCO và mô hình custom. COCO giữ vai trò bổ trợ nhận diện tổng quát nhờ tri thức học từ dữ liệu lớn, còn mô hình custom tập trung vào bốn lớp mục tiêu gồm dumbbell, flower, fruit và tree. Việc kết hợp hai nguồn dự đoán không chỉ nhằm tăng khả năng phát hiện mà còn nhằm giảm rủi ro bỏ sót khi một trong hai mô hình hoạt động chưa tốt ở các ảnh khó. Từ kết quả suy luận song song, hệ thống cần có bước hợp nhất detection theo nguyên tắc ưu tiên lớp và độ tin cậy để tạo tập kết quả đầu vào nhất quán cho giai đoạn hậu xử lý.

Một mục tiêu trọng tâm khác là phát triển chuỗi hậu xử lý theo ngữ cảnh để kiểm soát lỗi đếm. Trong các hệ thống chỉ dựa trên output thô của mô hình, đếm thừa do trùng lặp box và đếm thiếu do lọc quá mạnh là hai lỗi phổ biến, đặc biệt với lớp flower có kích thước nhỏ và mật độ dày. Vì vậy, đề tài đặt mục tiêu thiết kế rule theo điều kiện cảnh, chỉ kích hoạt khi phù hợp, thay vì áp cứng cho mọi trường hợp. Cách tiếp cận này giúp hệ thống cân bằng giữa giảm nhiễu và giữ đối tượng thật, đồng thời hạn chế hiện tượng regression khi tinh chỉnh tham số.

Bên cạnh phần mô hình và hậu xử lý, mục tiêu nghiên cứu còn bao gồm xây dựng cơ chế đánh giá định lượng xuyên suốt vòng đời phát triển. Hệ thống phải theo dõi đồng thời các chỉ số Precision, Recall, mAP50, mAP50-95 và MAE, kết hợp với log theo từng stage để truy vết nguyên nhân sai lệch. Mỗi lần thay đổi dữ liệu, mô hình hoặc rule đều cần có số liệu trước-sau để đối chiếu khách quan. Như vậy, mục tiêu nghiên cứu của đề tài hướng đến một pipeline hoàn chỉnh, có khả năng vận hành, kiểm chứng, cải tiến liên tục và mở rộng bền vững trong các giai đoạn tiếp theo.

1.2 Mục tiêu nghiên cứu

1.2.1 Mục tiêu chung

Mục tiêu chung của đồ án là xây dựng một hệ thống nhận diện và đếm vật thể trong ảnh có khả năng vận hành ổn định trên nhiều điều kiện dữ liệu thực tế, đồng thời đáp ứng yêu cầu ứng dụng về tốc độ xử lý và độ chính xác đầu ra. Hệ thống không được thiết kế theo hướng phụ thuộc vào một cấu hình cố định, mà theo kiến trúc mô-đun để thuận lợi cho việc mở rộng dữ liệu, thay thế mô hình và điều chỉnh quy tắc hậu xử lý trong các vòng phát triển tiếp theo. Cách tiếp cận này phù hợp với đặc thù của bài toán thị giác máy tính, nơi dữ liệu thường biến động theo bối cảnh thu thập và yêu cầu triển khai có thể thay đổi theo mục tiêu sử dụng.

Trọng tâm kỹ thuật của mục tiêu chung là cơ chế suy luận kết hợp giữa mô hình COCO và mô hình custom. Mô hình COCO cung cấp nền nhận diện tổng quát nhờ được huấn luyện trên dữ liệu lớn và đa dạng, trong khi mô hình custom được fine-tune từ dữ liệu Roboflow để tập trung vào bốn lớp mục tiêu của đề tài gồm

dumbbell, flower, fruit và tree. Việc khai thác đồng thời hai nguồn dự đoán giúp hệ thống tăng độ bao phủ phát hiện và giảm nguy cơ bỏ sót trong các tình huống ảnh phức tạp. Trên cơ sở đó, pipeline thực hiện hợp nhất kết quả theo nguyên tắc ưu tiên lớp và độ tin cậy nhằm tạo đầu vào nhất quán cho bước đếm.

Bên cạnh khả năng phát hiện, mục tiêu chung đặc biệt nhấn mạnh tính ổn định của kết quả đếm theo lớp. Trong dữ liệu thực tế, sai số đếm có thể tăng mạnh khi thay đổi ánh sáng, góc chụp, mật độ đối tượng hoặc mức độ che khuất. Do đó, hệ thống cần chuỗi hậu xử lý theo ngữ cảnh để kiểm soát lỗi trùng lặp bounding box, giảm nhầm lớp và hạn chế việc loại bỏ quá mức. Cơ chế này đóng vai trò quyết định trong việc cân bằng giữa giảm overcount và giảm undercount, nhất là với lớp flower có kích thước nhỏ và phân bố dày.

Ngoài ra, mục tiêu chung còn bao gồm xây dựng năng lực đánh giá định lượng và quản trị phiên bản kết quả. Hệ thống phải theo dõi đồng thời các chỉ số Precision, Recall, mAP50, mAP50-95 và MAE, đồng thời lưu log theo từng giai đoạn xử lý để phân tích nguyên nhân sai lệch. Việc duy trì cơ chế so sánh trước-sau sau mỗi lần điều chỉnh mô hình hoặc rule giúp quá trình cải tiến có căn cứ thực nghiệm rõ ràng, tránh tối ưu cảm tính. Từ đó, đồ án hướng tới một hệ thống nhận diện và đếm vật thể theo hướng toàn diện, có giá trị ứng dụng, có độ ổn định cao và có khả năng mở rộng bền vững.

1.2.2 Mục tiêu cụ thể

Để hiện thực hóa mục tiêu chung, đề tài xác định các mục tiêu cụ thể theo từng lớp công việc trong pipeline. Mục tiêu đầu tiên là chuẩn hóa tập dữ liệu phục vụ huấn luyện và đánh giá theo định dạng YOLO, bảo đảm đúng class id, không thiếu nhãn và không lệch cặp ảnh-nhãn. Việc chuẩn hóa không chỉ dừng ở cấu trúc thư mục mà còn bao gồm kiểm tra chất lượng nhãn, phát hiện alias không hợp lệ, thống kê phân bố lớp và nhận diện các trường hợp tiny box có nguy cơ gây sai lệch khi hậu xử lý. Đây là bước nền giúp mô hình học đúng tín hiệu dữ liệu và giảm lỗi lan truyền sang các giai đoạn sau.

Mục tiêu thứ hai là huấn luyện mô hình chính custo_all.pt bằng phương pháp fine-tune từ YOLO pretrained với cấu hình phù hợp tài nguyên phần cứng thực tế. Đề tài không đặt mục tiêu chạy cấu hình cực đại, mà ưu tiên thiết lập ổn định, có thể lặp lại và có khả năng triển khai liên tục theo phiên bản. Song song với đó, đề tài đặt mục tiêu tăng cường xử lý cho lớp khó flower thông qua dữ liệu flower-focused và cơ chế specialist trong những bối cảnh cần thiết. Việc phân tách xử lý lớp khó theo hướng chuyên sâu giúp cải thiện recall cục bộ mà vẫn giữ độ ổn định của các lớp còn lại.

Mục tiêu thứ ba là xây dựng pipeline suy luận hoàn chỉnh gồm chạy song song COCO và custom, hợp nhất detection theo ưu tiên lớp, áp dụng hậu xử lý theo ngữ cảnh và sinh kết quả đếm cuối cùng. Trong mục tiêu này, tính nhất quán giữa các bước được xem là yêu cầu bắt buộc, đặc biệt ở phần canonicalize định dạng detection và kiểm soát class order khi deploy. Hệ thống cũng phải có khả năng fallback trong các trường hợp rule quá mạnh làm rỗng kết quả, nhằm bảo đảm đầu ra không bị suy giảm bất thường ở các ca khó.

Mục tiêu thứ tư là thiết kế bộ đánh giá định lượng theo cả hai góc độ detection và counting. Ngoài các chỉ số chuẩn như Precision, Recall, mAP50 và mAP50-95, đề tài tập trung theo dõi MAE theo lớp, thống kê undercount-overcount và phân tích drop theo từng stage hậu xử lý. Việc duy trì các log phân tích giúp nhóm xác định chính xác nguyên nhân lỗi, từ đó điều chỉnh đúng điểm nghẽn thay vì nới ngưỡng toàn cục. Cuối cùng, đề tài đặt mục tiêu triển khai mô hình theo quy trình có kiểm soát metadata và version để bảo đảm tính truy vết, rollback và khả năng mở rộng hệ thống trong các vòng phát triển tiếp theo.

1.3 Phạm vi nghiên cứu

1.3.1 Không gian nghiên cứu

Không gian nghiên cứu của đồ án được xác định trong phạm vi nhận diện và đếm vật thể trên ảnh tĩnh. Đề tài tập trung vào bốn lớp mục tiêu gồm dumbbell, flower, fruit và tree. Hệ thống được phát triển trên môi trường Python, sử dụng YOLO cho cả huấn luyện và suy luận, đồng thời tổ chức dữ liệu và kết quả theo cấu trúc thư mục chuẩn để bảo đảm khả năng tái lập.

Về dữ liệu, nguồn chính được khai thác từ Roboflow, sau đó chuẩn hóa theo định dạng YOLO trước khi đưa vào huấn luyện. Tập dữ liệu được chia thành train và val để phục vụ đánh giá định lượng. Ngoài dữ liệu chuẩn, đề tài còn sử dụng các bộ kiểm thử nội bộ dạng hard-case, đặc biệt cho lớp flower, nhằm kiểm tra tính ổn định của hệ thống trong các tình huống ảnh khó như vật thể nhỏ, chồng lắp hoặc phân bố dày.

Về phạm vi kỹ thuật, đồ án tập trung vào kiến trúc YOLO kết hợp suy luận hybrid giữa mô hình COCO và mô hình custom, kèm theo hậu xử lý theo ngữ cảnh để giảm đếm thiếu, đếm thừa và nhầm lớp. Các hướng mở rộng như Faster R-CNN, Transformer, tracking video hoặc segmentation chưa nằm trong phạm vi hiện tại. Giới hạn này giúp đề tài giữ được tính khả thi trong thời gian thực hiện và tập trung đúng vào mục tiêu cốt lõi của hệ thống.

1.3.2 Thời gian nghiên cứu

Đồ án được triển khai theo các giai đoạn chính phù hợp với tiến độ học kỳ. Giai đoạn đầu tập trung khảo sát bài toán, xác định hướng tiếp cận kỹ thuật và lựa chọn kiến trúc phù hợp. Trong giai đoạn này, nhóm tìm hiểu các mô hình detection, hệ chỉ số đánh giá và yêu cầu của bài toán đếm để làm cơ sở thiết kế hệ thống.

Giai đoạn tiếp theo là thu thập và xử lý dữ liệu. Dữ liệu từ Roboflow được đưa về, chuẩn hóa theo định dạng YOLO, kiểm tra chất lượng nhãn và tổ chức lại theo cấu trúc train, val. Đây là bước có ý nghĩa quyết định vì dữ liệu sai nhãn, thiếu nhãn hoặc lệch cặp ảnh - nhãn sẽ ảnh hưởng trực tiếp đến chất lượng huấn luyện.

Sau khi dữ liệu ổn định, nhóm tiến hành huấn luyện mô hình chính theo hướng fine-tune từ YOLO pretrained, đồng thời theo dõi các chỉ số và lưu checkpoint để phục vụ so sánh phiên bản. Với lớp flower, hệ thống tiếp tục thử nghiệm hướng xử lý chuyên sâu để cải thiện kết quả trong các trường hợp khó.

Trên nền mô hình đã huấn luyện, giai đoạn kế tiếp tập trung xây dựng pipeline suy luận và hậu xử lý. Các quy tắc được thiết kế, kiểm thử và tinh chỉnh qua nhiều vòng đánh giá để giảm đếm thiếu, đếm thừa và nhầm lớp. Song song với đó, hệ thống được tích hợp vào ứng dụng app.py để kiểm tra khả năng vận hành trực tiếp trên dữ liệu đầu vào thực tế.

Giai đoạn cuối là đánh giá tổng thể, so sánh trước - sau khi điều chỉnh và hoàn thiện nội dung báo cáo. Các script đánh giá được sử dụng để tổng hợp các chỉ số Precision, Recall, mAP và MAE, từ đó rút ra kết luận và đề xuất cải tiến. Toàn bộ quy trình được thực hiện tuần tự trong thời gian học kỳ nhằm bảo đảm đủ thời lượng cho thực nghiệm, phân tích và hoàn thiện đồ án..

1.3.3 Phương pháp thực hiện

Phương pháp thực hiện đồ án được triển khai theo hướng thực nghiệm có kiểm soát, dựa trên ba trụ cột chính gồm dữ liệu, mô hình và hậu xử lý.Ở trụ cột dữ liệu, nguồn ảnh và nhãn được thu thập từ Roboflow, sau đchuẩn hóa theo định dạng YOLO trước khi đưa vào huấn luyện. Quy trình chuẩn hóa bao gồm kiểm tra class id, rà soát file nhãn lỗi, xử lý trường hợp thiếu cặp ảnh - nhãn và bảo đảm tính nhất quán giữa các tập train, val. Việc kiểm soát chất lượng dữ liệu ngay từ đầu giúp giảm nhiễu học, hạn chế sai lệch mô hình và tạo nền tảng ổn định cho toàn bộpipeline.

Ở trụ cột mô hình, đề tài sử dụng phương pháp fine-tune từ YOLO pretrained để tận dụng tri thức nền và rút ngắn thời gian hội tụ. Mô hình chính được huấn luyện với dữ liệu custom bốn lớp gồm dumbbell, flower, fruit và tree, trong đó các tham số như kích thước ảnh, batch size, số epoch và learning rate được lựa chọn

theo giới hạn phần cứng thực tế. Bên cạnh mô hình chính, lớp flower được xử lý theo hướng chuyên sâu bằng dữ liệu flower-focused và cơ chế specialist trong các tình huống khó, nhằm cải thiện độ nhạy nhận diện đối tượng nhỏ hoặc xuất hiện dày đặc.

Ở trụ cột suy luận và hậu xử lý, hệ thống áp dụng phương pháp hybrid bằng cách chạy song song mô hình COCO và mô hình custom. Kết quả từ hai nguồn được hợp nhất theo quy tắc ưu tiên lớp và mức tin cậy, sau đó đi qua chuỗi hậu xử lý gồm NMS theo lớp, loại trùng lặp và các quy tắc ngữ cảnh để giảm đếm thiếu, đếm thừa và nhầm lớp. Trong các trường hợp đặc biệt, cơ chế fallback được kích hoạt để tránh mất toàn bộ detections do ngưỡng lọc quá chặt. Toàn bộ rule được hiệu chỉnh qua nhiều vòng thử nghiệm trên tập hard-case nhằm bảo đảm cải thiện thực chất và hạn chế regression.

Cuối cùng, phương pháp đánh giá được xây dựng theo hướng định lượng để phản ánh đầy đủ hiệu quả của hệ thống. Nhóm chỉ số detection như Precision, Recall, mAP50, mAP50-95 được dùng để đo chất lượng nhận diện, trong khi MAE được dùng để đo chất lượng đếm. Kết quả đánh giá được lưu vết và so sánh giữa các phiên bản mô hình cũng như các lần điều chỉnh hậu xử lý, từ đó tạo cơ sở dữ liệu rõ ràng cho quá trình cải tiến liên tục.

1.3.4 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là bài toán phát hiện và đếm vật thể trên ảnh tĩnh trong điều kiện dữ liệu gần thực tế, với bốn lớp mục tiêu gồm dumbbell, flower, fruit và tree. Bài toán không chỉ yêu cầu xác định đúng vị trí và nhãn đối tượng mà còn cần thống kê chính xác số lượng theo từng lớp, do đó phạm vi nghiên cứu bao gồm cả detection và counting. Việc mở rộng từ nhận diện sang đếm làm tăng đáng kể độ phức tạp, vì hệ thống có thể đạt chỉ số phát hiện tương đối tốt nhưng vẫn cho sai số đếm lớn khi xuất hiện trùng lặp box, nhầm lớp hoặc bỏ sót đối tượng nhỏ.

Đề tài tập trung nghiên cứu đối tượng ở mức hệ thống, nghĩa là xem xét toàn bộ pipeline từ dữ liệu đến đầu ra cuối. Ở lớp dữ liệu, đối tượng nghiên cứu gồm cấu trúc tập ảnh-nhãn theo chuẩn YOLO, chất lượng gán nhãn, phân bố lớp và mức độ đại diện của các cảnh khó. Ở lớp mô hình, đối tượng nghiên cứu là cơ chế phối hợp giữa mô hình COCO và mô hình custom, khả năng bổ trợ giữa hai nguồn dự đoán và hiệu quả của chiến lược fine-tune theo lớp. Ở lớp xử lý đầu ra, đề tài nghiên cứu các quy tắc hợp nhất và hậu xử lý theo ngữ cảnh nhằm tối ưu độ ổn định khi đếm.

Một nội dung quan trọng trong đối tượng nghiên cứu là nhóm trường hợp khó của lớp flower. Lớp này có đặc điểm kích thước nhỏ, xuất hiện dày và dễ bị ảnh hưởng bởi nền ảnh, khiến hệ thống dễ rơi vào cả hai trạng thái đếm thiếu và đếm thừa. Vì vậy, đề tài xem đây là đối tượng trọng điểm để khảo sát tác động của rule,

specialist và chiến lược đánh giá hard-case. Việc phân tích sâu nhóm trường hợp này giúp làm rõ giới hạn của mô hình nền và vai trò thực tế của hậu xử lý theo ngữ cảnh.

Ngoài ra, đối tượng nghiên cứu còn bao gồm cơ chế vận hành thực nghiệm, cụ thể là khả năng lưu vết log, theo dõi stage statistics, đối chiếu phiên bản mô hình và kiểm soát class order khi triển khai. Đây là phần quan trọng để bảo đảm kết quả nghiên cứu có thể kiểm chứng và tái lập. Như vậy, đối tượng nghiên cứu của đề tài không chỉ là một mô hình nhận diện đơn lẻ, mà là một hệ thống phát hiện và đếm hoàn chỉnh có tính ứng dụng, có khả năng vận hành ổn định và có cơ sở cải tiến theo dữ liệu thực tế.

1.4 Công cụ và môi trường phát triển

Hệ thống được xây dựng trong môi trường Python, lựa chọn theo tiêu chí phổ biến, dễ tích hợp thư viện học sâu và thuận lợi cho việc mở rộng trong quá trình nghiên cứu. Thành phần cốt lõi của pipeline là Ultralytics YOLO, được sử dụng cho cả huấn luyện và suy luận mô hình. Bên cạnh đó, OpenCV được dùng để xử lý ảnh và trực quan hóa bounding box, trong khi NumPy và Pandas hỗ trợ xử lý dữ liệu, tổng hợp thống kê và phân tích kết quả thực nghiệm. Ở lớp giao diện, Streamlit được lựa chọn để xây dựng ứng dụng thử nghiệm giúp thao tác nạp ảnh, theo dõi kết quả đếm và kiểm tra nhanh các cấu hình suy luận.

Về tổ chức dữ liệu, hệ thống sử dụng định dạng YOLO với cấu trúc tách biệt train và val, mỗi ảnh đi kèm tệp nhãn tương ứng để bảo đảm tính nhất quán khi huấn luyện. Các script hỗ trợ như chuẩn hóa nhãn, audit dữ liệu, kiểm tra bias và phân tích failure mode được đưa vào pipeline nhằm giảm sai sót thủ công. Cách tổ chức này giúp dữ liệu được kiểm soát theo chu trình rõ ràng từ khâu thu thập đến triển khai, đồng thời cung cấp khả năng truy vết khi cần xác định nguyên nhân sai lệch giữa các phiên bản mô hình.

Về môi trường tính toán, quá trình huấn luyện và đánh giá được triển khai trên máy tính cá nhân có GPU tầm trung, phù hợp điều kiện thực tế của đồ án cơ sở. Do giới hạn phần cứng, cấu hình huấn luyện được điều chỉnh theo hướng cân bằng giữa hiệu năng và tính ổn định bộ nhớ, chẳng hạn sử dụng batch nhỏ, kích thước ảnh phù hợp và các cơ chế dừng sớm để tránh overfit không cần thiết. Ở giai đoạn suy luận, hệ thống ưu tiên cấu hình có thể chạy ổn định trên nhiều ảnh với thời gian xử lý hợp lý, thay vì tối ưu một chỉ số riêng lẻ.

Một thành phần quan trọng của môi trường phát triển là cơ chế quản trị phiên bản mô hình và metadata triển khai. Trọng số deploy, tệp metadata và log đánh giá được lưu theo cấu trúc chuẩn để bảo đảm khả năng rollback, đối chiếu và tái lập thí

nghiệm. Điều này giúp hệ thống không chỉ hoạt động ở mức demo, mà còn đáp ứng yêu cầu kiểm chứng kết quả trong báo cáo theo hướng thực nghiệm có kiểm soát.

1.5 Ý nghĩa của đề tài

Ý nghĩa khoa học của đề tài thể hiện ở việc tiếp cận bài toán nhận diện và đếm vật thể theo hướng hệ thống, thay vì chỉ tối ưu một mô hình phát hiện độc lập. Đề tài góp phần làm rõ rằng chất lượng đếm cuối cùng phụ thuộc đồng thời vào ba lớp thành phần gồm dữ liệu, mô hình và hậu xử lý. Việc kết hợp suy luận COCO-custom cùng cơ chế rule theo ngữ cảnh cung cấp một minh chứng thực nghiệm cho hướng phát triển pipeline nhiều tầng, trong đó mỗi tầng có vai trò riêng và có thể định lượng tác động thông qua log theo stage.

Ý nghĩa thực tiễn của đề tài nằm ở khả năng vận hành trên dữ liệu có biến động cao, nơi các phương pháp đơn giản thường cho kết quả thiếu ổn định. Hệ thống được thiết kế để vừa phát hiện vừa đếm theo lớp, phù hợp với nhu cầu kiểm kê và giám sát trong nhiều bối cảnh ứng dụng. Đặc biệt, cơ chế lưu vết kết quả, so sánh phiên bản và kiểm soát class order giúp giảm rủi ro vận hành, tạo điều kiện để hệ thống được sử dụng như một công cụ thử nghiệm có thể mở rộng theo yêu cầu thực tế.

Đề tài cũng có ý nghĩa phương pháp luận cho sinh viên trong quá trình thực hiện đồ án AI ứng dụng. Thay vì phụ thuộc hoàn toàn vào chỉ số mAP, nghiên cứu nhấn mạnh cách đánh giá kết hợp giữa detection và counting, đồng thời sử dụng bộ hard-case để kiểm tra độ bền của pipeline. Cách làm này giúp quá trình cải tiến có cơ sở khách quan, tránh tối ưu cảm tính và hạn chế tình trạng đạt kết quả cao trên tập dễ nhưng kém hiệu quả trên dữ liệu thực tế.

Ngoài ra, hệ thống được phát triển theo kiến trúc mô-đun, cho phép thay thế hoặc mở rộng từng thành phần mà không cần viết lại toàn bộ. Đây là nền tảng quan trọng để phát triển các hướng nghiên cứu tiếp theo như tăng cường specialist cho lớp khó, mở rộng dữ liệu theo miền ứng dụng, hoặc chuyển sang kịch bản video và theo dõi đối tượng theo thời gian. Vì vậy, đề tài không chỉ có giá trị hoàn thành mục tiêu học phần, mà còn tạo ra nền kỹ thuật có thể tiếp tục phát triển sau đồ án.

1.6 Bố cục của báo cáo

Báo cáo được tổ chức thành bảy chương theo trình tự từ cơ sở lý thuyết đến triển khai và đánh giá thực nghiệm. Chương 1 trình bày bối cảnh đề tài, mục tiêu nghiên cứu, phạm vi, phương pháp thực hiện, công cụ phát triển và ý nghĩa khoa học cũng như thực tiễn của hệ thống. Đây là chương đặt nền cho toàn bộ hướng tiếp cận của đồ án, đồng thời xác định rõ bài toán cần giải quyết và tiêu chí thành công của hệ thống.

Chương 2 tổng hợp cơ sở lý thuyết liên quan đến bài toán phát hiện và đếm vật thể, bao gồm vai trò của mô hình YOLO, mô hình COCO, nền tảng Streamlit và các chỉ số đánh giá. Nội dung chương này giúp làm rõ cơ sở lựa chọn công nghệ và cách đo lường hiệu quả, bảo đảm các quyết định kỹ thuật trong các chương sau có nền tảng học thuật rõ ràng.

Chương 3 trình bày phân tích và thiết kế hệ thống theo kiến trúc mô-đun, gồm luồng dữ liệu, các thành phần mô hình, cơ chế hợp nhất và lớp hậu xử lý. Chương 4 mô tả dữ liệu và tiền xử lý, tập trung vào chuẩn hóa nhãn, kiểm tra chất lượng dataset, quản lý phiên bản và tự động hóa bằng script để nâng tính nhất quán của pipeline.

Chương 5 trình bày quá trình huấn luyện mô hình, chiến lược fine-tune theo lớp, cơ chế deploy và quản trị metadata để bảo đảm khả năng vận hành và rollback. Chương 6 trình bày pipeline suy luận chi tiết từ chạy song song hai mô hình, hợp nhất detection, hậu xử lý theo ngữ cảnh đến đếm số lượng và ghi log phân tích. Chương 7 tổng hợp kết quả thử nghiệm, phân tích lỗi, đánh giá mức cải thiện so với baseline và đưa ra kết luận cùng hướng phát triển tiếp theo của đề tài.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Tổng quan bài toán phát hiện và đếm vật thể

2.1.1 Tổng quan

Bài toán phát hiện vật thể (Object Detection) là nền tảng quan trọng để thực hiện tác vụ đếm đối tượng trong ảnh. Tuy nhiên, trong phạm vi đề tài này, mục tiêu không chỉ dừng ở việc nhận diện đúng lớp dumbbell, flower, fruit, tree, mà còn phải bảo đảm số lượng dự đoán theo từng lớp bám sát thực tế. Vì vậy, yêu cầu của hệ thống cao hơn so với detection thuần túy. Trong nhiều trường hợp, mô hình có thể đạt mAP ở mức tốt nhưng đầu ra đếm vẫn thiếu ổn định do tồn tại các lỗi đặc trưng như trùng box, bỏ sót đối tượng nhỏ hoặc nhầm lẫn giữa các lớp có hình thái gần nhau. Thực nghiệm của dự án cho thấy lớp flower là ví dụ điển hình vì có kích thước đa dạng, mật độ dày và dễ hòa lẫn vào nền ảnh, dẫn đến sai số đếm rõ rệt nếu chỉ dựa vào kết quả detection thô.

Khác với cách tiếp cận một mô hình đơn lẻ, hệ thống của đề tài triển khai pipeline suy luận kết hợp giữa mô hình COCO và mô hình custom. Cách thiết kế này giúp tăng độ bao phủ phát hiện trong các bối cảnh ảnh phức tạp, nhưng đồng thời làm phát sinh yêu cầu hợp nhất kết quả để tránh trùng lặp. Do đó, bài toán đếm trong dự án được hiểu theo nghĩa rộng: không chỉ dự đoán đối tượng, mà còn xử lý xung đột giữa nhiều nguồn dự đoán để tạo ra tập detections cuối cùng đủ ổn định cho bước thống kê. Nói cách khác, chất lượng đếm không chỉ phụ thuộc vào năng lực của một mô hình, mà phụ thuộc vào toàn bộ chuỗi xử lý từ dự đoán đến tinh lọc.

Trong thực tế triển khai, hệ thống thường gặp ba nhóm sai số chính. Nhóm thứ nhất là undercount, xuất hiện khi vật thể nhỏ, bị che khuất hoặc có độ tương phản thấp khiến mô hình bỏ sót. Nhóm thứ hai là overcount, phát sinh khi cùng một đối tượng bị sinh nhiều bounding box chồng lắp. Nhóm thứ ba là nhầm lớp chéo (cross-class confusion), thường gặp trong các cảnh có nền phức tạp, ví dụ flower bị nhầm sang fruit hoặc tree. Các sai số này cho thấy việc tối ưu mAP chưa đủ để bảo đảm chất lượng đếm, đặc biệt khi hệ thống cần vận hành trên dữ liệu đa dạng ngoài môi trường huấn luyện.

Để kiểm soát các sai lệch trên, đề tài áp dụng chuỗi hậu xử lý theo ngữ cảnh sau bước hợp nhất dự đoán. Chuỗi này bao gồm lọc theo ngưỡng tin cậy, NMS theo lớp, loại trùng lặp theo IoU, xử lý ưu tiên lớp, và áp dụng các quy tắc chuyên biệt cho trường hợp khó như ảnh có mật độ flower cao. Bên cạnh đó, cơ chế fallback được sử dụng để giảm nguy cơ mất toàn bộ kết quả trong các tình huống rule lọc quá chặt. Tất cả các bước này đều hướng đến mục tiêu thống nhất là tăng độ ổn định của đầu ra đếm thay vì chỉ cải thiện riêng từng chỉ số detection.

Từ góc nhìn ứng dụng, hệ thống không chỉ trả về kết quả dự đoán tức thời mà còn phải hỗ trợ theo dõi và phân tích qua logging. Đây là điểm khác biệt quan trọng giữa bài toán nghiên cứu thuần detection và bài toán triển khai thực tế trong đề tài. Vì vậy, nền tảng lý thuyết cần được mở rộng theo hướng pipeline counting hoàn chỉnh: detection -> hợp nhất -> hậu xử lý -> counting -> đánh giá. Cách tiếp cận này giúp giải thích rõ nguyên nhân sai số theo từng giai đoạn, đồng thời tạo cơ sở kỹ thuật vững chắc cho các bước tối ưu ở những phần tiếp theo của báo cáo.

2.1.2 Tổng quan mô hình YOLO và vai trò trong dự án YOLO

YOLO là nhóm mô hình phát hiện vật thể một giai đoạn, được thiết kế để cân bằng giữa tốc độ suy luận và độ chính xác. Khác với các phương pháp hai giai đoạn truyền thống, YOLO dự đoán trực tiếp vị trí và nhãn đối tượng trong một lần lan truyền, vì vậy phù hợp với các bài toán cần phản hồi nhanh và triển khai trên tài nguyên phần cứng phổ thông. Trong phạm vi đề tài, đặc điểm này giúp hệ thống đáp ứng yêu cầu xử lý ảnh đầu vào ổn định mà vẫn giữ được chất lượng phát hiện trên các lớp mục tiêu.

Vai trò của YOLO trong hệ thống được thể hiện ở hai tầng. Tầng thứ nhất là mô hình custom fine-tune từ checkpoint pretrained để nhận diện bốn lớp dumbbell, flower, fruit và tree theo đúng mục tiêu của đề tài. Tầng thứ hai là sự tương thích đầu ra với pipeline hậu xử lý và đếm, nhờ cấu trúc dự đoán dạng bounding box kèm độ tin cậy rõ ràng. Điều này giúp hệ thống dễ thực hiện các bước hợp nhất detection, lọc trùng và điều chỉnh theo ngữ cảnh trước khi thống kê số lượng cuối cùng.

Bên cạnh ưu điểm, đề tài cũng ghi nhận giới hạn thực tế của YOLO khi áp dụng trên ảnh có mật độ đối tượng cao hoặc vật thể nhỏ, điển hình là lớp flower. Trong các trường hợp này, dự đoán thô có thể xuất hiện trùng lặp hoặc bỏ sót, làm sai lệch kết quả đếm nếu sử dụng trực tiếp. Vì vậy, hệ thống không dừng ở output của mô hình mà kết hợp thêm cơ chế fusion và hậu xử lý theo điều kiện cảnh để ổn định đầu ra.

Từ góc độ triển khai, YOLO giữ vai trò là nền phát hiện chính, còn chất lượng đếm cuối phụ thuộc vào sự phối hợp giữa mô hình, dữ liệu và quy tắc xử lý sau suy luận. Cách tổ chức này phù hợp với định hướng của đồ án, trong đó mục tiêu không chỉ là đạt chỉ số detection tốt mà còn tạo ra kết quả thống kê có thể sử dụng trong thực tế.

2.1.3 Nền tảng Streamlit và vai trò trong hệ thống Streamlit

Streamlit là nền tảng Python hỗ trợ xây dựng giao diện tương tác cho ứng dụng dữ liệu và học máy theo hướng nhanh, gọn và dễ mở rộng. Trong đề tài này, Streamlit được sử dụng để tạo lớp giao tiếp giữa người dùng và pipeline suy luận,

cho phép nạp ảnh đầu vào, chạy mô hình, quan sát kết quả trực quan và theo dõi số lượng theo từng lớp. Việc có giao diện vận hành giúp hệ thống vượt ra khỏi phạm vi script thử nghiệm đơn lẻ và tiến gần hơn tới một sản phẩm có thể trình diễn, kiểm thử và đánh giá liên tục.

Vai trò quan trọng của Streamlit nằm ở khả năng rút ngắn vòng lặp tối ưu. Thay vì chỉnh tham số trực tiếp trong mã nguồn rồi chạy lại toàn bộ, người dùng có thể thay đổi các cấu hình suy luận như confidence, IoU, max_det hoặc kích thước ảnh ngay trên giao diện. Cách tương tác này giúp kiểm tra nhanh tác động của từng tham số lên chất lượng đếm và phát hiện các trường hợp lỗi trong thời gian ngắn hơn.

Ngoài chức năng hiển thị, Streamlit còn hỗ trợ quy trình lưu vết thực nghiệm thông qua cơ chế ghi log kết quả và lưu ảnh đã gắn bounding box. Đây là thành phần cần thiết cho phần đánh giá định lượng, vì nhóm có thể đối chiếu trực tiếp giữa số liệu và hiện trạng box trên ảnh để xác định nguyên nhân sai lệch. Khi kết hợp với log theo stage trong pipeline hậu xử lý, Streamlit trở thành công cụ kiểm thử rất hiệu quả trong các vòng tinh chỉnh.

Từ góc nhìn kiến trúc, Streamlit đóng vai trò lớp frontend nhẹ, kết nối trực tiếp với backend suy luận và các script phân tích. Sự kết hợp này tạo nên một quy trình hoàn chỉnh từ đầu vào đến đầu ra, góp phần nâng cao tính thực tiễn và khả năng mở rộng của toàn bộ hệ thống.

2.2 Các tiêu chí đánh giá

Trong đề tài này, chất lượng hệ thống không được đánh giá theo một chỉ số đơn lẻ mà được xem xét trên hai góc độ bổ trợ nhau: chất lượng phát hiện đối tượng (detection) và chất lượng đếm (counting). Cách đánh giá này xuất phát từ mục tiêu thực tế của hệ thống là không chỉ phát hiện đúng lớp mà còn trả về số lượng tin cậy theo từng lớp. Vì vậy, một mô hình có mAP cao nhưng đếm sai nhiều vẫn chưa đáp ứng yêu cầu của bài toán.

Nhóm chỉ số detection gồm Precision, Recall, mAP50 và mAP50-95. Precision phản ánh tỷ lệ dự đoán đúng trên tổng số dự đoán, qua đó cho thấy mức độ nhiễu của hệ thống. Recall phản ánh tỷ lệ phát hiện đúng trên tổng số đối tượng thực, thể hiện khả năng bao phủ của mô hình trong các tình huống khó. mAP50 và mAP50-95 là các chỉ số tổng hợp đánh giá chất lượng định vị bounding box ở các ngưỡng IoU khác nhau. Trong quá trình thực nghiệm, nhóm sử dụng các chỉ số này để theo dõi độ ổn định giữa các phiên bản mô hình, đặc biệt khi so sánh trước và sau các lần fine-tune hoặc thay đổi dữ liệu.

Nhóm chỉ số counting giữ vai trò trung tâm trong phần đánh giá đầu ra. Chỉ số chính được sử dụng là MAE (Mean Absolute Error), tính theo sai lệch tuyệt đối trung bình giữa số lượng dự đoán và số lượng thực tế. MAE càng thấp thì chất lượng đếm càng tốt. Ngoài giá trị tổng, MAE còn được tính theo từng lớp để xác định lớp khó cần ưu tiên cải tiến. Trong bối cảnh dự án, lớp flower thường có mức sai số cao hơn do mật độ dày và kích thước đối tượng nhỏ, vì vậy việc theo dõi MAE theo lớp giúp định hướng hiệu chỉnh đúng trọng tâm.

Bên cạnh MAE, hệ thống còn theo dõi hai xu hướng sai số là undercount và overcount. Undercount phản ánh số lần dự đoán thấp hơn thực tế, thường liên quan đến ngưỡng lọc cao hoặc bỏ sót đối tượng nhỏ. Overcount phản ánh số lần dự đoán cao hơn thực tế, thường do trùng box hoặc nhạy quá mức ở các cảnh nhiễu. Việc tách hai dạng sai số này giúp nhóm đánh giá bản chất lỗi rõ hơn thay vì chỉ nhìn một con số tổng hợp. Trên cơ sở đó, các quyết định kỹ thuật như nới hoặc siết confidence, điều chỉnh IoU hoặc sửa rule hậu xử lý có thể được thực hiện có căn cứ hơn.

Một tiêu chí bổ sung quan trọng trong dự án là theo dõi mức giảm đối tượng qua từng stage hậu xử lý, còn gọi là stage drop. Các mốc như after_nms, after_resolve_overlap, after_flower_pruning hoặc after_scene_rules cho phép xác định bước nào đang loại bỏ quá nhiều đối tượng thật. Đây là cách đánh giá đặc thù của pipeline hybrid có nhiều tầng xử lý, giúp phát hiện sớm regression sau mỗi lần chỉnh rule. Việc theo dõi stage drop đặc biệt hữu ích với lớp flower vì lớp này nhạy với các quy tắc lọc dày đặc.

Ngoài chất lượng dự đoán, hệ thống cũng ghi nhận thời gian xử lý (latency) như một tiêu chí phụ để bảo đảm khả năng vận hành. Mặc dù đồ án không đặt mục tiêu real-time tuyệt đối, tốc độ suy luận vẫn cần ở mức hợp lý để hỗ trợ kiểm thử liên tục và trình diễn ứng dụng. Do đó, độ trễ được theo dõi cùng với nhóm chỉ số chất lượng để bảo đảm hệ thống đạt được sự cân bằng giữa độ chính xác và hiệu năng.

Nhờ kết hợp các tiêu chí detection, counting, phân tích xu hướng lỗi và theo dõi stage drop, hệ thống đánh giá trong đề tài phản ánh được cả chiều rộng lẫn chiều sâu của chất lượng đầu ra. Cách tiếp cận này giúp quá trình cải tiến mô hình và hậu xử lý dựa trên dữ liệu thực nghiệm, đồng thời tạo cơ sở rõ ràng cho các kết luận kỹ thuật của đồ án.

2.3 Định hướng đề tài

Các nghiên cứu gần đây cho thấy bài toán phát hiện vật thể đã chuyển từ mục tiêu nhận diện thuần túy sang hướng vận hành thực tế, trong đó yêu cầu về độ ổn định, khả năng giải thích và tốc độ xử lý ngày càng quan trọng. Mankodiya et al.

(2022) nhấn mạnh vai trò của mô hình có khả năng giải thích trong bối cảnh hệ thống tự hành, cho thấy kết quả phát hiện không chỉ cần chính xác mà còn cần minh bạch để hỗ trợ phân tích quyết định. Ở góc nhìn ứng dụng rộng hơn, Nawaz et al. (2022) tổng hợp xu hướng phát hiện vật thể trong viễn thám, đa phương tiện và nông nghiệp, qua đó khẳng định rằng chất lượng dữ liệu và kịch bản triển khai thực tế quyết định mạnh đến hiệu quả mô hình. Bên cạnh đó, Matsuzaka và Yashiro (2023) chỉ ra rằng các kỹ thuật thị giác máy tính cần được đặt trong cấu trúc hệ thống hoàn chỉnh, có kết hợp logic chuyên gia để xử lý các trường hợp khó mà mô hình học sâu đơn lẻ còn hạn chế.

Về mặt kiến trúc, Terven et al. (2023) cho thấy họ YOLO tiếp tục là lựa chọn phù hợp cho các bài toán yêu cầu cân bằng giữa tốc độ và độ chính xác, đặc biệt khi cần triển khai trên tài nguyên phần cứng giới hạn. Kết quả này phù hợp với lựa chọn kỹ thuật của đề tài khi sử dụng YOLO làm nền huấn luyện và suy luận. Ngoài ra, Ahmad et al. (2020) cho thấy việc tinh chỉnh kiến trúc và tham số có thể cải thiện đáng kể hiệu quả phát hiện trong bối cảnh dữ liệu không lý tưởng. Ở hướng mới hơn, Silva et al. (2025) gợi ý tiềm năng mở rộng sang các framework kết hợp theo chuỗi thời gian để tăng khả năng phát hiện trong môi trường phức tạp. Nhìn chung, các công trình liên quan xác nhận rằng một hệ thống phát hiện hiệu quả cần sự phối hợp giữa mô hình mạnh, dữ liệu phù hợp và chiến lược hậu xử lý có kiểm soát, thay vì phụ thuộc vào một thành phần đơn lẻ.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Mô tả hệ thống

Hệ thống trong đề tài được thiết kế theo mô hình pipeline tổng thể để giải quyết đồng thời hai yêu cầu: phát hiện đúng đối tượng và đếm đúng số lượng theo lớp. Khác với cách triển khai một mô hình đơn lẻ, kiến trúc của hệ thống kết hợp song song hai nguồn suy luận gồm mô hình COCO và mô hình custom bốn lớp (dumbbell, flower, fruit, tree). COCO giữ vai trò tăng khả năng tổng quát trong các bối cảnh ảnh phức tạp, còn custom giữ vai trò chuyên biệt cho lớp mục tiêu của bài toán. Sự kết hợp này giúp hệ thống cân bằng giữa độ bao phủ phát hiện và độ chính xác theo ngữ cảnh dữ liệu thực tế.

Về luồng xử lý, hệ thống bắt đầu từ ảnh đầu vào do người dùng cung cấp qua giao diện ứng dụng. Ảnh sau đó đi qua bước tiền xử lý và được đưa vào hai nhánh suy luận chạy song song. Kết quả từ mỗi nhánh được chuẩn hóa về cùng cấu trúc dữ liệu để phục vụ bước hợp nhất. Tại bước hợp nhất, hệ thống xử lý trùng lặp và xung đột dự đoán theo nguyên tắc ưu tiên lớp, ưu tiên nguồn và mức độ tin cậy. Sau hợp nhất, kết quả tiếp tục được đưa qua tầng hậu xử lý nhiều bước nhằm giảm nhiễu, ổn định số box hợp lệ và hạn chế sai số đếm. Cuối cùng, hệ thống tổng hợp số lượng theo từng lớp, hiển thị kết quả trực quan trên Streamlit và lưu log để phục vụ phân tích.

Điểm nổi bật của hệ thống là tầng hậu xử lý theo ngữ cảnh, đặc biệt với lớp flower. Trong các cảnh dày đặc, nếu dùng kết quả detection thô trực tiếp sẽ dễ phát sinh overcount do box chồng lắp. Vì vậy pipeline áp dụng các quy tắc tinh lọc theo mật độ, kích thước và mức chồng lắp để giảm đếm thừa nhưng vẫn giữ đối tượng thật. Ngược lại, với ảnh thưa hoặc đối tượng nhỏ, hệ thống nới điều kiện lọc để giảm nguy cơ undercount. Ngoài ra, cơ chế fallback được thiết kế nhằm tránh tình trạng đầu ra rỗng khi chuỗi rule quá chặt. Cách xử lý này thể hiện rõ định hướng thực tiễn của đề tài, vì chất lượng đếm cuối cùng được kiểm soát bởi toàn bộ pipeline chứ không chỉ bởi mô hình suy luận ban đầu.

Hệ thống cũng hỗ trợ mở rộng theo hướng specialist cho lớp khó. Cơ chế này cho phép bổ sung mô hình chuyên gia để cải thiện recall cục bộ mà không làm phá vỡ cấu trúc tổng thể. Với dự án hiện tại, hướng specialist tập trung vào flower trong các tình huống mật độ cao hoặc đối tượng nhỏ. Việc tách specialist khỏi mô hình chính giúp giảm rủi ro ảnh hưởng tiêu cực lên các lớp còn lại, đồng thời tạo nền tảng để mở rộng cho các lớp khó khác khi dữ liệu tăng theo thời gian.

Về tổ chức tài nguyên, hệ thống được bố trí theo cấu trúc thư mục rõ ràng gồm dữ liệu, trọng số mô hình, script tiền xử lý và script đánh giá. Cách tổ chức này hỗ trợ tốt cho tái lập thực nghiệm, truy vết phiên bản và kiểm chứng kết quả trước-sau khi điều chỉnh rule hoặc mô hình. Nhờ đó, hệ thống trong đề tài không chỉ là một mô hình AI độc lập mà là một kiến trúc hoàn chỉnh gồm dữ liệu, suy luận, hậu xử lý, đánh giá và giao diện vận hành.

3.2 Kiến trúc tổng thể

Kiến trúc tổng thể của hệ thống được xây dựng theo hướng mô-đun nhằm bảo đảm tính linh hoạt trong phát triển và khả năng bảo trì trong quá trình vận hành. Toàn bộ pipeline được tổ chức thành bốn lớp chức năng chính gồm lớp dữ liệu, lớp mô hình, lớp hậu xử lý và lớp giao diện. Mỗi lớp đảm nhiệm một nhiệm vụ riêng nhưng liên kết chặt chẽ theo chuỗi xử lý thống nhất, từ đầu vào ảnh đến đầu ra đếm theo lớp. Cách tổ chức này cho phép tối ưu từng thành phần mà không làm ảnh hưởng toàn bộ hệ thống, phù hợp với đặc thù đồ án có nhiều vòng lặp thử nghiệm và cải tiến.

Lớp dữ liệu chịu trách nhiệm quản lý tài nguyên huấn luyện và kiểm định, bao gồm tập train, val theo chuẩn YOLO, dữ liệu chuyên sâu cho lớp khó và các bộ kiểm thử hard-case. Vai trò của lớp này không chỉ là lưu trữ mà còn kiểm soát chất lượng dữ liệu thông qua các bước chuẩn hóa và audit. Các lỗi như sai class id, lệch cặp ảnh - nhãn, file nhãn rỗng hoặc bounding box không hợp lệ cần được phát hiện sớm trước khi đưa vào huấn luyện. Việc kiểm soát dữ liệu theo lớp riêng biệt giúp hệ thống giảm lỗi lan truyền sang các bước phía sau và nâng độ ổn định của mô hình.

Lớp mô hình là nơi thực hiện suy luận đa nguồn với hai thành phần chính: mô hình COCO pretrained và mô hình custom fine-tune theo bài toán. COCO giữ vai trò bổ trợ tổng quát trong các bối cảnh khó, còn custom tập trung vào bốn lớp mục tiêu của hệ thống. Trong trường hợp cần tăng độ nhạy cho lớp khó, kiến trúc cho phép mở rộng thêm specialist mà không thay đổi cấu trúc lõi. Cách phân tách vai trò mô hình giúp tận dụng ưu điểm từng nguồn dự đoán và hạn chế rủi ro khi một mô hình đơn lẻ hoạt động chưa ổn định.

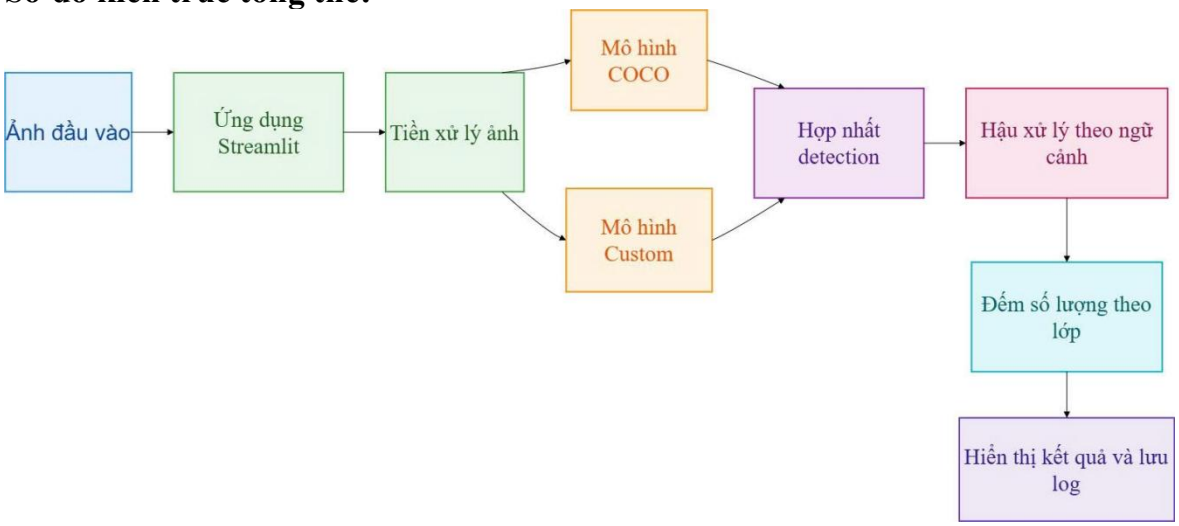
Lớp hậu xử lý là thành phần quyết định chất lượng đếm cuối cùng. Sau khi nhận kết quả từ các mô hình, hệ thống thực hiện hợp nhất detections, lọc trùng theo IoU, xử lý xung đột nhãn, kiểm soát confidence theo ngữ cảnh và áp dụng các quy tắc chuyên biệt cho từng lớp. Với lớp flower, các rule được tinh chỉnh theo mật độ để giảm overcount ở cảnh dày và hạn chế undercount ở cảnh thưa. Ngoài ra, cơ chế fallback và các kiểm tra nhất quán được tích hợp để tránh mất toàn bộ đối tượng khi

rule lọc quá chặt. Nhờ đó, hệ thống không chỉ dựa vào dự đoán thô mà có thêm tầng ổn định hóa đầu ra theo mục tiêu đếm.

Lớp giao diện được triển khai bằng Streamlit, đóng vai trò điểm tương tác giữa người dùng và pipeline suy luận. Giao diện cho phép nạp ảnh, chọn tham số, theo dõi kết quả trực quan, quan sát thống kê theo lớp và lưu vết thực nghiệm phục vụ đánh giá. Việc có lớp giao diện riêng giúp quá trình kiểm thử diễn ra nhanh, đồng thời hỗ trợ trình diễn hệ thống trong bối cảnh báo cáo đồ án.

Về quan hệ xử lý, hệ thống vận hành theo luồng một chiều dữ liệu -> mô hình -> hậu xử lý -> giao diện. Tuy nhiên, ở mức nghiên cứu có thêm vòng phản hồi: kết quả từ giao diện và log đánh giá được dùng để điều chỉnh dữ liệu, mô hình và quy tắc hậu xử lý ở các vòng tiếp theo. Về triển khai kỹ thuật, app.py giữ vai trò điều phối trung tâm, còn các script hỗ trợ trong thư mục scripts đảm nhiệm các tác vụ audit dữ liệu, đánh giá mô hình và phân tích lỗi. Cấu trúc này giúp hệ thống vừa có thể chạy trực tiếp, vừa thuận lợi cho nghiên cứu và cải tiến liên tục.

Sơ đồ kiên trúc tổng thể:



Hình 3.1 Sơ đồ kiên trúc tổng thể

3.3 Các thành phần chính

3.3.1 Ứng dụng suy luận và hiển thị (app.py)

Trong kiến trúc của đề tài, app.py là thành phần điều phối trung tâm, giữ vai trò kết nối dữ liệu đầu vào, mô hình suy luận, hậu xử lý và giao diện vận hành. Khác với cách tổ chức tách rời giữa script thử nghiệm và ứng dụng trình diễn, app.py trong dự án được xây dựng như một pipeline hoàn chỉnh có khả năng chạy trực tiếp trên ảnh thực tế. Thành phần này tiếp nhận ảnh đầu vào, kích hoạt luồng suy luận hybrid giữa COCO và custom, xử lý hợp nhất kết quả, áp dụng các quy tắc hậu xử lý, sau đó trả về kết quả hiển thị và thống kê số lượng theo lớp. Cách tổ chức tập trung giúp toàn hệ thống hoạt động nhất quán và thuận lợi cho kiểm thử lặp lại.

Ở tầng tương tác người dùng, app.py sử dụng Streamlit để cung cấp giao diện trực quan và dễ thao tác. Người dùng có thể tải ảnh, chọn nguồn dữ liệu phù hợp và điều chỉnh các tham số suy luận quan trọng như confidence, IoU cho NMS, kích thước ảnh và giới hạn số lượng detections. Việc thay đổi tham số trực tiếp trên giao diện giúp rút ngắn vòng lặp thử nghiệm so với phương pháp chạy script dòng lệnh, đồng thời hỗ trợ đánh giá nhanh tác động của từng cấu hình lên kết quả đếm. Với bài toán của đề tài, khả năng quan sát ảnh đã gắn bounding box kèm thống kê theo lớp là yếu tố quan trọng để phát hiện sớm các lỗi overcount, undercount hoặc nhầm lớp.

Ở tầng xử lý kỹ thuật, app.py không chỉ gọi mô hình mà còn chuẩn hóa kết quả dự đoán trước khi đưa vào hợp nhất. Detections từ COCO và custom được đưa về cùng cấu trúc dữ liệu để thuận tiện cho xử lý trùng lặp, ưu tiên lớp và kiểm soát xung đột nhãn. Sau hợp nhất, chuỗi hậu xử lý theo ngữ cảnh được áp dụng để ổn định đầu ra, đặc biệt cho các tình huống khó của lớp flower. Các bước như lọc trùng theo IoU, xử lý nhầm lớp chéo, tinh chỉnh ngưỡng theo bối cảnh và fallback khi cần thiết đều được tích hợp trong app.py để bảo đảm chất lượng đếm cuối cùng không phụ thuộc vào một bước đơn lẻ.

Ngoài chức năng suy luận và hiển thị, app.py còn đóng vai trò thu thập và quản lý kết quả thực nghiệm. Hệ thống lưu thông tin dự đoán, thống kê lớp, và dữ liệu lịch sử để phục vụ phân tích lỗi và so sánh giữa các lần chạy. Nhờ cơ chế logging này, quá trình cải tiến mô hình và hậu xử lý có cơ sở định lượng rõ ràng, hạn chế điều chỉnh cảm tính. Đây là điểm quan trọng giúp đề tài duy trì tính khoa học trong đánh giá, đồng thời hỗ trợ truy vết khi phát sinh sai lệch sau các lần cập nhật.

Với cách tổ chức như trên, app.py vừa là bộ điều phối kỹ thuật của toàn bộ pipeline, vừa là lớp tương tác phục vụ kiểm thử và trình diễn. Thành phần này giúp kết nối các khối dữ liệu, mô hình, hậu xử lý và đánh giá thành một hệ thống thống nhất có khả năng vận hành thực tế, phù hợp mục tiêu đồ án là xây dựng giải pháp hoàn chỉnh thay vì chỉ dừng ở mức mô hình nhận diện đơn thuần.

3.3.2 Mô hình COCO (phát hiện đối tượng phổ biến)

Trong hệ thống của đề tài, mô hình COCO được sử dụng như nguồn suy luận tổng quát đã pretrained trên bộ dữ liệu lớn và đa dạng. Vai trò chính của thành phần này không phải thay thế mô hình custom ở bốn lớp mục tiêu, mà là bổ trợ trong các trường hợp ảnh khó, nơi mô hình custom có thể bỏ sót hoặc suy giảm độ tin cậy. Nhờ nền tri thức thị giác rộng, COCO thường giữ được khả năng phát hiện ổn định hơn khi điều kiện đầu vào biến động mạnh về ánh sáng, góc chụp hoặc mức độ nhiễu nền.

Khi thực hiện suy luận, COCO tạo ra tập detections gồm nhãn lớp, độ tin cậy và tọa độ bounding box. Tập kết quả này không được dùng trực tiếp để đếm mà được đưa qua bước chuẩn hóa trước khi hợp nhất với kết quả từ mô hình custom. Quá trình chuẩn hóa giúp đồng nhất định dạng dữ liệu giữa hai nguồn dự đoán, từ đó giảm lỗi kỹ thuật khi xử lý trùng lặp hoặc xung đột nhãn ở các bước phía sau. Việc chuẩn hóa cũng là điều kiện quan trọng để các rule hậu xử lý hoạt động nhất quán trong toàn pipeline.

Ở bước hợp nhất, hệ thống áp dụng nguyên tắc ưu tiên theo lớp mục tiêu và mức tin cậy. Trong trường hợp COCO và custom cùng dự đoán một vùng đối tượng tương tự, kết quả custom thường được ưu tiên để giữ tính chuyên biệt cho bài toán. Ngược lại, nếu COCO phát hiện được đối tượng mà custom không có và kết quả đó thỏa điều kiện kiểm soát của pipeline, hệ thống có thể giữ lại như một nguồn bổ sung nhằm giảm bỏ sót. Cách xử lý này giúp cân bằng giữa tính đặc thù của custom và tính bao phủ tổng quát của COCO

Việc tích hợp COCO trong kiến trúc hybrid mang lại giá trị rõ ràng về độ ổn định đầu ra. Khi dữ liệu custom chưa bao phủ hết các tình huống thực tế hoặc xuất hiện mẫu ảnh ngoài miền huấn luyện, COCO đóng vai trò lớp hỗ trợ quan trọng để duy trì khả năng phát hiện. Nhờ đó, hệ thống giảm rủi ro suy giảm chất lượng ở các ca khó và tạo nền tốt hơn cho bước hậu xử lý trước khi chốt số đếm cuối cùng.

3.3.3 Mô hình custom (dumbbell/flower/fruit/tree)

Mô hình custom là thành phần cốt lõi của hệ thống và giữ vai trò nguồn dự đoán chính cho bài toán nhận diện, đếm vật thể theo lớp. Mô hình này được xây dựng bằng phương pháp fine-tune từ YOLO pretrained nhằm tận dụng đặc trưng thị giác nền, sau đó chuyên biệt hóa cho bốn lớp mục tiêu của đề tài gồm dumbbell, flower, fruit và tree. Trong kiến trúc vận hành hiện tại, custom là mô hình được triển khai chính thức và có ảnh hưởng trực tiếp đến chất lượng kết quả đếm cuối cùng.

Về dữ liệu huấn luyện, mô hình custom sử dụng tập dữ liệu đã được chuẩn hóa từ Roboflow theo định dạng YOLO, bảo đảm tính nhất quán giữa ảnh và nhãn trước khi đưa vào quá trình train. Cấu hình huấn luyện được lựa chọn theo điều kiện phần cứng thực tế nhằm duy trì khả năng chạy ổn định và có thể tái lập, với các tham số trọng yếu như kích thước ảnh, batch size và số epoch được điều chỉnh phù hợp. Sau khi đạt tiêu chí chất lượng, trọng số tốt nhất được triển khai vào đường dẫn model_custom/weights/custo_all.pt và đi kèm metadata để kiểm soát class order khi suy luận.

Trong pipeline hybrid, kết quả từ mô hình custom được ưu tiên ở các lớp mục tiêu khi hợp nhất với COCO. Nguyên tắc này phản ánh đúng định hướng của hệ

thống: tối ưu chất lượng cho các lớp nghiệp vụ cụ thể thay vì mở rộng quá nhiều lớp ngoài phạm vi đề tài. Việc đặt custom ở vị trí ưu tiên cũng giúp giảm xung đột nhãn trong bước hợp nhất, đồng thời giữ tính nhất quán giữa kết quả dự đoán và mục tiêu đánh giá của đồ án.

Ngoài vai trò suy luận chính, mô hình custom còn là trung tâm của quá trình cải tiến kỹ thuật. Mọi thay đổi về dữ liệu, cấu hình huấn luyện hoặc quy tắc hậu xử lý đều được đo lường thông qua tác động lên chất lượng đầu ra của custom. Vì vậy, đây không chỉ là thành phần triển khai, mà còn là điểm tham chiếu quan trọng để theo dõi tiến bộ hệ thống qua từng phiên bản.

3.3.4 Cơ chế kết hợp hai mô hình (fusion + ưu tiên lớp)

Cơ chế fusion là thành phần trung tâm trong kiến trúc hybrid của hệ thống, vì chất lượng đầu ra cuối cùng phụ thuộc trực tiếp vào cách hợp nhất dự đoán từ COCO và custom. Nếu chỉ chạy song song hai mô hình mà không có quy tắc hợp nhất rõ ràng, hệ thống rất dễ phát sinh trùng lặp, xung đột nhãn và sai lệch số đếm. Do đó, thiết kế fusion của đề tài được xây dựng theo hai nguyên tắc cốt lõi: bảo toàn tính chuyên biệt của lớp mục tiêu và giảm tối đa trùng box đa nguồn.

Về quy trình, fusion bắt đầu từ bước chuẩn hóa detections giữa hai mô hình về cùng cấu trúc dữ liệu, bao gồm tên lớp, độ tin cậy, tọa độ bounding box và nguồn dự đoán. Sau khi chuẩn hóa, hệ thống thực hiện kiểm tra chồng lắp theo IoU để xác định các cặp có khả năng trùng đối tượng. Trong trường hợp custom và COCO cùng dự đoán trên một vùng tương đương, kết quả custom được ưu tiên nhằm bảo đảm đầu ra bám sát mục tiêu bốn lớp của đề tài. Khi hai detection cùng lớp và cùng nguồn có mức chồng lắp cao, hệ thống giữ detection có confidence tốt hơn để tránh đếm thừa.

Đối với các trường hợp xung đột khác lớp, fusion không xử lý cứng theo một điều kiện duy nhất mà kết hợp ưu tiên lớp và chênh lệch độ tin cậy. Cách tiếp cận này đặc biệt cần thiết ở các cặp dễ nhầm như flower - fruit hoặc flower - tree trong ảnh nền phức tạp. Nếu custom không đưa ra dự đoán đủ tin cậy ở một vùng, kết quả COCO có thể được giữ lại theo ngưỡng kiểm soát để giảm nguy cơ bỏ sót. Nhờ đó, fusion vừa tránh thiên lệch tuyệt đối về một nguồn, vừa duy trì được tính ổn định trong các bối cảnh khó.

Giá trị chính của cơ chế fusion là đồng thời giảm overcount và undercount trước khi bước vào hậu xử lý chi tiết. Overcount giảm nhờ loại trùng giữa hai nguồn, còn undercount giảm nhờ khả năng bổ sung dự đoán trong trường hợp custom chưa bao phủ đủ. Điều này cho thấy fusion trong hệ thống không đơn thuần là ghép kết quả, mà là một lớp ra quyết định trung gian có vai trò trực tiếp trong việc nâng chất lượng đếm cuối cùng.

3.3.5 Hậu xử lý và sinh kết quả cuối

Hậu xử lý là tầng kỹ thuật cuối trước khi hệ thống thực hiện đếm và trả kết quả cho người dùng. Trong kiến trúc của đề tài, đây không phải bước phụ trợ mà là thành phần quyết định độ ổn định của đầu ra, đặc biệt trong các tình huống ảnh khó có mật độ đối tượng cao hoặc xuất hiện nhầm lớp chéo. Sau khi hoàn tất fusion giữa COCO và custom, hệ thống đưa tập detections qua chuỗi xử lý nhiều tầng để loại nhiễu, giảm trùng lặp và chuẩn hóa lại theo ngữ cảnh trước khi chốt số lượng theo lớp.

Các kỹ thuật hậu xử lý được áp dụng theo thứ tự rõ ràng. Trước hết, NMS theo lớp được dùng để loại bỏ các box chồng lắp cơ bản. Tiếp theo, hệ thống chạy các rule chuyên biệt cho lớp flower nhằm xử lý hai lỗi đặc trưng là overcount trong cảnh dày và undercount trong cảnh thưa. Song song với đó, cơ chế suppress cross-class confusion được dùng để giảm xung đột giữa các cặp lớp dễ nhầm như flower - fruit hoặc flower - tree. Trong các trường hợp chuỗi rule quá chặt làm suy giảm mạnh kết quả, cơ chế fallback được kích hoạt để tránh mất toàn bộ đối tượng hợp lệ.

Sau khi hoàn tất hậu xử lý, hệ thống thực hiện tổng hợp số lượng theo từng lớp và sinh đầu ra cuối cùng gồm ba thành phần chính. Thành phần thứ nhất là ảnh đã gắn bounding box để kiểm tra trực quan. Thành phần thứ hai là bảng thống kê số lượng theo lớp phục vụ mục tiêu đếm. Thành phần thứ ba là log JSON lưu thông tin dự đoán và trạng thái xử lý để hỗ trợ phân tích offline. Cách tổ chức đầu ra như vậy giúp người dùng vừa quan sát nhanh kết quả, vừa có dữ liệu định lượng để đối chiếu khi cần kiểm thử sâu.

Một điểm quan trọng của tầng hậu xử lý trong đề tài là tính thích nghi theo vòng lặp cải tiến. Các rule không được xem là cố định mà được hiệu chỉnh liên tục dựa trên dữ liệu lỗi thực tế và kết quả đánh giá định lượng. Nhờ cơ chế cập nhật có kiểm soát này, pipeline có khả năng nâng dần chất lượng qua từng phiên bản, thay vì phụ thuộc hoàn toàn vào việc thay đổi mô hình học sâu. Chính yếu tố đó giúp hệ thống duy trì tính ổn định cao hơn trong vận hành thực tế.

CHƯƠNG 4: DỮ LIỆU VÀ TIỀN XỬ LÝ

4.1 Cấu trúc dataset và tổ chức tài nguyên

Tập dữ liệu chính của hệ thống được tổ chức trong thư mục model_custom/dataset theo chuẩn YOLO, với cấu trúc tách biệt rõ ràng giữa train và val. Mỗi tập con đều gồm hai nhánh images và labels, trong đó mỗi ảnh phải có file nhãn .txt tương ứng cùng tên. Nội dung nhãn tuân theo định dạng YOLO gồm class_id, x_center, y_center, width, height, được chuẩn hóa theo kích thước ảnh gốc. Cách tổ chức này giúp pipeline huấn luyện và suy luận hoạt động đồng nhất, đồng thời giảm nguy cơ phát sinh lỗi khi mở rộng dữ liệu.

Hệ thống sử dụng cố định thứ tự lớp trong toàn bộ vòng đời dự án, gồm 0=dumbbell, 1=flower, 2=fruit, 3=tree. Việc giữ class order nhất quán giữa dữ liệu, mô hình và metadata triển khai là yêu cầu bắt buộc để tránh lỗi ánh xạ nhãn sau khi deploy. Khi bổ sung dữ liệu mới, ảnh và nhãn được đưa đúng vào thư mục tương ứng của train hoặc val, kèm bước kiểm tra trùng tên và đồng bộ cặp ảnh - nhãn trước khi đưa vào huấn luyện. Quy trình này giúp bảo đảm dữ liệu mở rộng không làm phá vỡ cấu trúc hiện có.

Ngoài ảnh và nhãn, lớp dữ liệu còn gồm các tài nguyên quản trị phục vụ kiểm soát chất lượng. File data.yaml giữ thông tin đường dẫn train, val và bảng tên lớp. Các file báo cáo audit như dataset_label_audit.json được dùng để theo dõi số lượng ảnh, tổng đối tượng, nhãn bất thường, tiny bbox, class lỗi và danh sách mẫu nghi ngờ cần rà soát thủ công. Bên cạnh đó, các báo cáo kiểm tra thiên lệch val như val_bias_check.json hỗ trợ đánh giá phân bố dữ liệu và mức ổn định của mô hình trên tập kiểm định.

Việc tổ chức đầy đủ dữ liệu, metadata và log kiểm tra giúp hệ thống duy trì tính truy vết theo phiên bản trong suốt quá trình nghiên cứu. Nhờ đó, mỗi lần huấn luyện hoặc tinh chỉnh đều có thể đối chiếu lại nguồn dữ liệu đã dùng, tạo nền tảng vững cho đánh giá định lượng và cải tiến liên tục.

4.2 Normalize nhãn và xử lý alias

Để tránh sai lệch giữa dữ liệu nhãn trên đĩa và tên lớp dùng trong suy luận, hệ thống áp dụng quy trình chuẩn hóa nhãn ngay từ giai đoạn tiền xử lý. Mục tiêu của bước này là đưa toàn bộ tên lớp về một bộ quy ước thống nhất, loại bỏ khác biệt do alias, dấu tiếng Việt hoặc kiểu ghi không đồng nhất giữa các nguồn dữ liệu.

Trong dự án, script scripts/normalize_flower_labels.py được sử dụng để chuẩn hóa nhãn, đặc biệt với nhóm dữ liệu flower-focused. Các alias như cam, cam canh được ánh xạ về orange; chuoi được ánh xạ về banana; hoa về flower; cay về tree.

Đồng thời, tên lớp được đưa về chữ thường và bỏ dấu để giảm rủi ro sai khớp trong quá trình huấn luyện và suy luận.

Bên cạnh ánh xạ alias, script còn kiểm tra định dạng file nhãn theo chuẩn YOLO và chỉ giữ các dòng hợp lệ gồm 5 cột. Quy trình này giúp hạn chế lỗi do thiếu cột, dư cột hoặc sai định dạng tọa độ. Nhờ đó, dữ liệu đầu vào cho mô hình giữ được tính nhất quán, giảm nhiễu học và giảm nguy cơ phát sinh lỗi hệ thống ở các bước phía sau.

Ở tầng suy luận, app.py tiếp tục chuẩn hóa tên lớp thông qua các hàm normalize_class_name và canonicalize_final_detections trước khi áp dụng rule hậu xử lý. Việc đồng bộ tên lớp giữa dữ liệu, metadata và runtime giúp các cấu trúc như CONFUSION_PAIRS và CLASS_PRIORITY hoạt động chính xác, đồng thời hạn chế lỗi class order mismatch khi kiểm tra bằng check_class_order_and_warn(). Cách làm này bảo đảm pipeline giữ được tính nhất quán từ dữ liệu huấn luyện đến kết quả đầu ra.

4.3 Dataset hoa chuyên sâu & augmentation

Trong hệ thống của đề tài, flower là lớp có độ khó cao nhất do kích thước đối tượng nhỏ, mật độ xuất hiện dày và dễ chồng lắp trong cùng một khung ảnh. Vì vậy, dự án xây dựng một nhánh dữ liệu chuyên sâu cho lớp này thay vì chỉ dựa vào bộ dữ liệu tổng quát ban đầu. Quy trình được thực hiện qua script scripts/prepare_flower_dataset.py với mục tiêu tăng độ bao phủ dữ liệu theo đúng các tình huống lỗi thường gặp trong thực tế.

Dữ liệu flower được chia theo mức mật độ cảnh để áp dụng chiến lược tăng cường phù hợp. Nhóm sparse gồm các ảnh có tối đa 2 đối tượng hoa, nhóm medium gồm 3 đến 4 đối tượng, và nhóm dense gồm từ 5 đối tượng trở lên. Với nhóm sparse, hệ thống áp dụng oversampling thông qua lật ngang và xoay ảnh ở các góc 90, 180, 270 độ nhằm tạo thêm biến thể hình học, giúp mô hình học tốt hơn trong các trường hợp góc chụp thay đổi. Các bản sao được lưu với hậu tố tên rõ ràng như _flip hoặc _rot90 để thuận tiện truy vết và kiểm tra.

Với nhóm dense, dữ liệu được xử lý theo hướng chia tile 640 x 640 có overlap 0.2 để tăng khả năng giữ lại đối tượng nhỏ khi huấn luyện. Trong quá trình cắt tile, hệ thống chỉ giữ những bounding box thỏa điều kiện tỷ lệ che phủ tối thiểu, nhằm tránh đưa vào các nhãn méo hoặc thiếu thông tin. Cách xử lý này giúp cải thiện đáng kể chất lượng dữ liệu cho các cảnh dày, nơi mô hình thường gặp lỗi undercount hoặc nhầm box.

Toàn bộ quá trình augmentation được ghi lại trong manifest JSON, bao gồm số lượng mẫu tạo thêm theo từng nhóm như train_sparse_augmented, train_

medium_augmented và train_dense_tiles. Manifest đóng vai trò quan trọng trong việc theo dõi coverage dữ liệu, phục vụ đánh giá chất lượng trước và sau khi tinh chỉnh pipeline, đồng thời hỗ trợ ra quyết định khi huấn luyện specialist.

Tập dữ liệu flower chuyên sâu được dùng cho huấn luyện mô hình specialist flower_best.pt với cấu hình ưu tiên độ nhạy đối tượng nhỏ, thường dùng kích thước ảnh lớn hơn, batch thấp và số epoch phù hợp với tài nguyên. Nhờ đó, hệ thống có thêm một nguồn hỗ trợ cho các ca khó của lớp flower, tạo nền tảng để giảm MAE và tăng tính ổn định đếm trong các vòng cải tiến tiếp theo.

4.4 Kiểm tra chất lượng nhãn & monitoring

Kiểm tra chất lượng nhãn trong dự án được triển khai như một quy trình lặp lại theo từng phiên bản dữ liệu, không phải tác vụ thực hiện một lần rồi kết thúc. Mỗi khi dữ liệu được cập nhật hoặc mở rộng, hệ thống đều chạy lại bước audit để bảo đảm tính nhất quán trước khi huấn luyện. Quy trình này được thực hiện thông qua script scripts/dataset_label_audit.py và áp dụng trên toàn bộ thư mục labels của tập dữ liệu hiện hành.

Kết quả audit ghi nhận đầy đủ các chỉ số quan trọng như tổng số ảnh, tổng số file nhãn, số file nhãn rỗng, lỗi định dạng bounding box, số lượng tiny bbox, class không hợp lệ, ảnh không có nhãn và nhãn không có ảnh tương ứng. Khi phát hiện alias không chuẩn hoặc class vượt ngoài phạm vi 0 đến 3, hệ thống đưa các mẫu này vào danh sách suspicious_files kèm mô tả lỗi cụ thể để nhóm kiểm tra thủ công. Cách làm này giúp phát hiện sớm các lỗi ngầm có thể gây sai lệch lớn trong quá trình huấn luyện và suy luận.

Bên cạnh kiểm tra lỗi cấu trúc, audit còn theo dõi phân bố lớp để cảnh báo mất cân bằng dữ liệu. Nếu một lớp chiếm tỷ trọng quá cao so với phần còn lại, hệ thống ghi nhận cảnh báo imbalance để nhóm cân nhắc bổ sung dữ liệu hoặc áp dụng oversampling cho các lớp yếu. Đây là bước quan trọng vì dữ liệu lệch phân bố thường làm mô hình thiên vị lớp phổ biến, dẫn đến giảm recall ở các lớp có ít mẫu hơn.

Một nội dung giám sát khác là theo dõi tiny bbox theo ngưỡng diện tích tương đối. Khi số lượng tiny bbox tăng cao, nguy cơ nhiễu nhãn và sai số hậu xử lý cũng tăng tương ứng, đặc biệt trong các rule prune theo ngữ cảnh. Vì vậy, các mẫu tiny bbox cần được đánh dấu để rà soát, giữ lại có kiểm soát hoặc loại bỏ nếu không còn giá trị huấn luyện. Tất cả kết quả audit được lưu theo từng phiên bản dữ liệu, tạo thành chuỗi lịch sử chất lượng có thể truy vết, so sánh và dùng làm căn cứ kỹ thuật trong quá trình viết báo cáo.

4.5 Roboflow & version tracking

Roboflow là nguồn dữ liệu gốc của đề tài. Mỗi lần tải bộ dữ liệu mới, nhóm đều lưu đầy đủ thông tin gồm tên bộ dữ liệu, phiên bản, ngày tải và tổng số ảnh. Dữ liệu gốc được giữ trong thư mục roboflow_exports để bảo toàn trạng thái ban đầu. Sau bước này, dữ liệu chỉ được đưa vào model_custom/dataset khi đã hoàn thành chuẩn hóa nhãn và kiểm tra chất lượng. Cách làm này giúp tách bạch rõ giữa dữ liệu thô và dữ liệu đã xử lý, đồng thời hỗ trợ truy vết nguồn gốc từng ảnh khi cần đối chiếu trong quá trình đánh giá.

Để quản lý lịch sử cập nhật, dự án duy trì tệp DATASET_CHANGELOG.md hoặc phần tương đương trong README. Mỗi lần thay đổi đều có ghi chú theo mốc thời gian và nội dung cụ thể, ví dụ bổ sung ảnh tree từ một phiên bản Roboflow nhất định hoặc thêm dữ liệu dumbbell theo từng đợt. Nhật ký này có vai trò quan trọng trong báo cáo vì cho phép liên hệ trực tiếp giữa thay đổi dữ liệu và biến động kết quả huấn luyện. Nhờ đó, nhóm có thể xác định chính xác phiên bản dữ liệu đã dùng cho từng lần chạy mô hình và từng script thực nghiệm.

Với nhánh dữ liệu chuyên sâu cho flower, việc theo dõi phiên bản được thực hiện thêm qua tệp dataset_flower_focused/manifest.json. Tệp manifest lưu số lượng mẫu theo các mức mật độ như sparse, medium và dense, đồng thời ghi nhận số ảnh validation khó. Thông tin này đặc biệt hữu ích khi cần huấn luyện lại mô hình specialist để cải thiện recall, vì nhóm có thể biết rõ dữ liệu nào đã tham gia ở từng giai đoạn. Cách quản lý này giúp quá trình thực nghiệm có tính lặp lại, dễ kiểm chứng và phù hợp với yêu cầu của đồ án theo hướng triển khai thực tế.

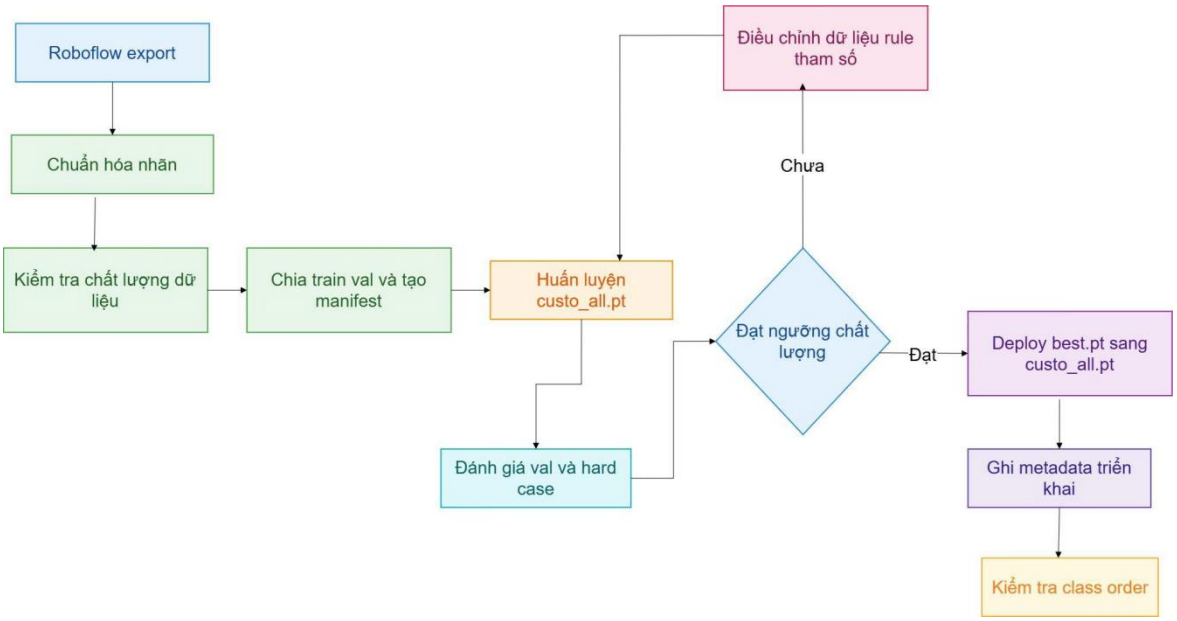
4.6 Automation & script-driven pipeline

Để bảo đảm tính nhất quán khi cập nhật dữ liệu và giảm lỗi thao tác thủ công, hệ thống được tổ chức theo một pipeline tự động dựa trên các script chuyên trách. Trong đó, normalize_flower_labels.py thực hiện chuẩn hóa tên lớp và định dạng nhãn, prepare_flower_dataset.py xây dựng tập dữ liệu flower-focused theo kịch bản đã thiết kế, dataset_label_audit.py kiểm tra tính hợp lệ của cặp ảnh - nhãn, check_val_bias.py đánh giá nhanh thiên lệch trên tập validation, còn analyze_flower_failure_modes.py hỗ trợ phân tích lỗi theo từng nhóm tình huống khó. Chuỗi xử lý này tạo thành quy trình chuẩn gồm normalize, audit, tạo manifest và lưu log, nhờ vậy mỗi lần bổ sung dữ liệu đều đi qua cùng một tiêu chuẩn kiểm soát chất lượng.

Một điểm quan trọng của cách tiếp cận script-driven là khả năng tái lập thực nghiệm. Thay vì xử lý thủ công theo từng bước rời rạc, toàn bộ quy trình được chạy theo thứ tự rõ ràng và sinh ra tệp kết quả trung gian dưới dạng JSON hoặc CSV. Các tệp này đóng vai trò bằng chứng kỹ thuật, cho phép đối chiếu giữa các lần chạy

và hỗ trợ truy vết khi phát sinh sai lệch kết quả huấn luyện. Cấu trúc này đặc biệt phù hợp với đồ án theo hướng thực nghiệm vì mọi thay đổi đều có dữ liệu đi kèm để kiểm chứng.

Ngoài vai trò chuẩn hóa dữ liệu, pipeline script còn kết nối trực tiếp với tầng suy luận trong app.py để phục vụ kiểm thử và đánh giá. Ví dụ, hoa_test_log.json lưu thông tin suy luận ở các ảnh hard-case khi bật chế độ debug và append log, từ đó cung cấp đầu vào đáng tin cậy cho phần phân tích kết quả ở chương đánh giá. Nhờ cơ chế tự động hóa này, hệ thống không chỉ vận hành ổn định hơn mà còn nâng cao độ tin cậy của toàn bộ quá trình từ chuẩn bị dữ liệu, huấn luyện đến triển khai.



Hình 4.1 Sơ đồ pipeline dữ liệu và huấn luyện

CHƯƠNG 5: MÔ HÌNH HUẤN LUYỆN

5.1 Huấn luyện mô hình chính (custo_all.pt)

Mô hình chính của hệ thống được xây dựng theo hướng fine-tune YOLO trên tập dữ liệu gồm bốn lớp mục tiêu: dumbbell, flower, fruit và tree. Dữ liệu huấn luyện được tổ chức trong thư mục model_custom/dataset theo chuẩn YOLO, chia rõ train và val, trong đó mỗi tệp nhãn chứa class_id cùng các tham số bounding box đã chuẩn hóa theo kích thước ảnh. Trước khi huấn luyện, tập dữ liệu được kiểm tra để bảo đảm không lệch cặp ảnh - nhãn, không sai class_id và không phát sinh lỗi định dạng. Cách chuẩn bị này giúp mô hình học đúng phân bố dữ liệu thực tế và hạn chế lỗi lan sang các giai đoạn suy luận, hậu xử lý.

Do điều kiện phần cứng sử dụng GPU GTX 1650 4 GB, cấu hình huấn luyện được tối ưu theo hướng ổn định bộ nhớ nhưng vẫn giữ chất lượng học. Cụ thể, batch size được đặt nhỏ trong khoảng 1 đến 2, kích thước ảnh đầu vào 640 và số epoch là 40, kết hợp patience bằng 10 để dừng sớm khi mô hình không còn cải thiện rõ rệt, qua đó giảm nguy cơ overfit. Các hệ số loss được điều chỉnh theo mục tiêu bài toán đếm, gồm cls bằng 1.5, box bằng 7.5 và dfl bằng 1.5 nhằm tăng vai trò phân lớp và định vị. Đồng thời, các kỹ thuật tăng cường dữ liệu như mosaic 0.6 và mixup 0.15 được áp dụng để mở rộng độ đa dạng mẫu và cải thiện khả năng tổng quát của mô hình trong các điều kiện ảnh khác nhau.

Trong quá trình huấn luyện, hệ thống ghi lại đầy đủ kết quả để phục vụ theo dõi và đối chiếu giữa các phiên bản. Tệp results.csv lưu diễn biến loss và các chỉ số đánh giá, các ảnh train_batch và val_batch hỗ trợ kiểm tra trực quan chất lượng nhãn và xu hướng học, còn checkpoint được lưu theo từng epoch trong _flower_runs/<run>/weights với các tệp epoch{n}.pt, last.pt và best.pt. Khi mô hình đạt mức chất lượng chấp nhận được, với Precision xấp xỉ 0.71, Recall xấp xỉ 0.39 và mAP50 xấp xỉ 0.41, quy trình deploy được thực hiện qua lệnh train_flower_focused.py--deploy để sao chép best.pt về model _custom/weights/custo_all.pt. Đồng thời hệ thống tạo tệp custo_all.flower_ focused.deployment.json chứa thông tin run, đường dẫn data.yaml, manifest và bộ siêu tham số, giúp app.py nạp đúng class order và bảo toàn ngữ cảnh huấn luyện khi triển khai thực tế.

5.2 Deploy & metadata governance

Sau khi hoàn tất huấn luyện, trọng số tốt nhất best.pt được triển khai về đường dẫn model_custom/weights/custo_all.pt để dùng cho suy luận trong ứng dụng. Trước khi ghi đè, hệ thống luôn tạo bản sao dự phòng custo_all.pt.bak nhằm bảo toàn phiên bản đang chạy, giúp giảm rủi ro khi bản mới phát sinh lỗi ngoài mong muốn. Cùng thời điểm deploy, hàm write_deploy_metadata() sinh tệp custo_all.flower_focused.deployment.json để lưu toàn bộ thông tin cấu hình của lần phát hành, bao gồm thời điểm deploy, run nguồn, đường dẫn checkpoint tốt nhất, data yaml, manifest, chế độ chạy và các siêu tham số chính. Nhờ đó, mỗi phiên bản mô hình đều có hồ sơ kỹ thuật đi kèm, thuận lợi cho truy vết và kiểm chứng trong báo cáo.

Tệp metadata được app.py đọc trực tiếp khi khởi động để hiển thị thông tin phiên bản ở thanh sidebar. Cơ chế này giúp người vận hành xác nhận ngay mô hình đang dùng thuộc bản build nào, tránh tình trạng chạy nhầm checkpoint hoặc nhầm cấu hình dữ liệu. Trong trường hợp cần quay lại phiên bản ổn định trước đó, quy trình rollback được thực hiện đơn giản bằng cách phục hồi tệp custo_all.pt.bak và đồng bộ lại metadata tương ứng. Cách tổ chức này bảo đảm quá trình chuyển phiên bản diễn ra nhanh, rõ ràng và có thể lặp lại.

Ngoài vai trò lưu vết, metadata còn là lớp kiểm soát an toàn cho toàn bộ pipeline suy luận. Hàm check_class_order_and_warn() đối chiếu class_names giữa dataset hiện tại và metadata deployment; nếu phát hiện sai khác, ứng dụng sẽ cảnh báo và không cho deploy mặc định. Cơ chế này đặc biệt cần thiết khi dữ liệu được cập nhật theo đợt, ví dụ phát sinh thay đổi tên lớp hoặc alias như cam, chuoi mà chưa đồng bộ thứ tự lớp trong cấu hình triển khai. Nhờ kiểm tra này, hệ thống ngăn sớm lỗi class mismatch, bảo đảm kết quả nhận diện và đếm luôn nhất quán với dữ liệu huấn luyện.

5.3 Chiến lược fine-tune theo lớp và rule context

Chiến lược huấn luyện của đề tài không áp dụng một cấu hình cố định cho toàn bộ lớp, mà điều chỉnh theo tình trạng dữ liệu thực tế của từng nhóm đối tượng. Khi dữ liệu tree hoặc dumbbell được mở rộng, hệ thống thực hiện gộp dữ liệu bằng script merge_tree_datasets.py hoặc bổ sung trực tiếp vào tập huấn luyện, sau đó fine-tune thêm một số epoch với cấu hình freeze=6 để giữ ổn định backbone và tập trung cập nhật head phân lớp. Cách làm này giúp mô hình học nhanh các mẫu mới mà không làm suy giảm mạnh năng lực đã có trên các lớp còn lại, đồng thời phù hợp với điều kiện phần cứng giới hạn.

Trong giai đoạn suy luận, lớp tree được xử lý theo cơ chế ngữ cảnh để hạn chế hiện tượng bị loại bỏ quá mức trong cảnh dày đối tượng. Cụ thể, khi phát hiện bối

cảnh tree-heavy với tree_n lớn hơn hoặc bằng ngưỡng đặt trước, cờ tree_hard_scene được kích hoạt để nới các điều kiện lọc, ví dụ giảm ngưỡng confidence từ 0.92 xuống 0.75 và giảm ngưỡng area từ 0.30 xuống 0.10. Nhờ đó, các bounding box tree hợp lệ được giữ lại nhiều hơn thay vì bị suppress sớm. Ngược lại, khi số lượng flower nhỏ, nhánh suppress weak tree được bỏ qua để tránh tác động không cần thiết lên lớp tree trong các ảnh không phải bối cảnh hoa dày. Quy tắc này được kết hợp với logic tree needs love trong scene rules nhằm cân bằng giữa giảm nhiễu và bảo toàn recall.

Đối với lớp flower, hệ thống áp dụng nhóm quy tắc chuyên biệt vì đây là lớp khó nhất trong bài toán đếm. Các bước prune và collapse chỉ được kích hoạt khi mật độ hoa đủ cao, chẳng hạn flower_count từ 5 trở lên và tỉ lệ flower_ratio lớn hơn 0.65, nhằm xử lý hiện tượng trùng lặp trong cảnh dense mà không làm giảm chất lượng ở cảnh thưa. Với các hoa rất nhỏ có area_ratio dưới 0.0003, hệ thống vẫn giữ dự đoán nếu confidence đạt từ 0.45 trở lên để hạn chế đếm thiếu ở nhóm tiny object. Việc chỉ bật rule trong đúng bối cảnh giúp pipeline tránh can thiệp toàn cục, giảm nguy cơ regression khi cập nhật mô hình hoặc mở rộng dữ liệu ở các lớp khác.

5.4 Cấu hình huấn luyện và tài nguyên

Quá trình huấn luyện được thực hiện bằng script train_flower_focused.py với cấu hình chính gồm kích thước ảnh 640, batch 2, 40 epoch, workers 0, device 0 và patience 10. Bộ siêu tham số được điều chỉnh theo mục tiêu nhận diện và đếm, trong đó hệ số loss cho phân lớp, định vị và dfl lần lượt là 1.5, 7.5 và 1.5. Đồng thời, hệ thống áp dụng các kỹ thuật tăng cường dữ liệu như mosaic 0.6, mixup 0.15, kết hợp amp, cos_lr và close_mosaic ở giai đoạn cuối để ổn định hội tụ. Cấu hình này giúp mô hình duy trì khả năng tổng quát mà vẫn phù hợp với điều kiện phần cứng hiện có.

Do sử dụng GPU 4 GB, tài nguyên huấn luyện được quản lý theo hướng ưu tiên tính ổn định. Khi bộ nhớ gần chạm ngưỡng, có thể giảm kích thước ảnh xuống 512 hoặc hạ batch về 1 để tránh lỗi tràn bộ nhớ. Trong trường hợp OOM vẫn xuất hiện, các kỹ thuật tăng cường nặng như mixup hoặc mosaic sẽ được tắt tạm thời để bảo đảm phiên huấn luyện tiếp tục chạy. Cách điều chỉnh theo tầng này giúp tiết kiệm thời gian thử sai và phù hợp với môi trường thực nghiệm của đồ án.

Trong suốt quá trình train, kết quả được theo dõi qua results.csv với các trường thời gian, loss, metric và learning rate. Nhóm đồng thời sử dụng TensorBoard hoặc Streamlit để quan sát xu hướng hội tụ theo từng epoch. Các ảnh train_batch và val_batch đóng vai trò kiểm tra trực quan, hỗ trợ phát hiện sớm lỗi nhãn hoặc hiện tượng học lệch. Với mô hình specialist cho flower, cấu hình được đẩy lên imgsz 960 và tăng max_det trong khoảng 260 đến 320 để cải thiện khả năng

bắt hoa nhỏ. Bên cạnh đó, cơ chế run_tiled_flower_detections được dùng để mở rộng vùng quét trên ảnh dày đối tượng, giúp tăng recall trong hard-case. Sau mỗi lần chạy, toàn bộ artifacts gồm results.csv, weights/best.pt và val_batch đều được lưu lại nhằm so sánh giữa các phiên bản và phục vụ phân tích ở chương đánh giá.

5.5 Đánh giá training & benchmarking

Sau mỗi lần huấn luyện, mô hình được đánh giá bằng các script chuyên dụng như evaluate_custo_all.py và compare_old_best_dumbbell.py để thu thập các chỉ số cốt lõi gồm Precision, Recall, mAP và MAE. Việc đánh giá không dừng ở một lần chạy đơn lẻ mà được thực hiện lặp lại theo từng phiên bản candidate nhằm so sánh trực tiếp với baseline. Cách tổ chức này giúp nhóm theo dõi rõ xu hướng cải thiện hoặc suy giảm của mô hình sau mỗi thay đổi về dữ liệu, tham số hoặc rule hậu xử lý.

Đối với lớp flower, nhóm sử dụng thêm flower_eval_improvement.py để phân tích theo từng nhóm bối cảnh, đặc biệt là sparse và dense. Script này tổng hợp chênh lệch giữa baseline và candidate theo các tiêu chí quan trọng như MAE, số ảnh dự đoán bằng 0, undercount và overcount. Nhờ tách riêng theo mật độ đối tượng, kết quả đánh giá phản ánh sát thực tế hơn so với chỉ nhìn một chỉ số tổng. Đây là cơ sở quan trọng để quyết định nên ưu tiên tăng recall hay siết nhiễu trong các vòng tinh chỉnh tiếp theo.

Bên cạnh tập đánh giá tổng quát đã được đề ra ở trên, hệ thống còn duy trì bộ hard-20 cho flower và đôngf thời lưu kết quả trong hoa_test_log.json ngay tại thư mục tmp_test_outputs/hoa_test_after_recover. Tệp log này ghi chi tiết TP, FP, FN cùng các chỉ số tổng hợp, ví dụ Precision khoảng 0.63, Recall xấp xỉ 0.38 và MAE tương ứng theo từng lần chạy. Bộ hard-20 được xem là chuẩn nội bộ để kiểm tra nhanh tác động của các thay đổi rule, vì nó tập trung vào đúng nhóm ảnh khó gây dao động kết quả đếm.

Ngoài các metric truyền thống, pipeline còn theo dõi stage statistics qua từng bước hậu xử lý như after_nms, after_flower_pruning và after_scene_rules, đồng thời ghi vào infer_log.json khi bật chế độ debug. Script trace_finalize_steps.py được dùng để truy vết số lượng box bị giảm ở từng stage, qua đó xác định chính xác rule nào đang làm mất detection nhiều nhất. Khi nới ngưỡng hoặc kích hoạt specialist, nhóm chạy lại bộ hard-20 để cập nhật MAE và drop stats, bảo đảm mọi quyết định tối ưu đều có căn cứ định lượng rõ ràng.

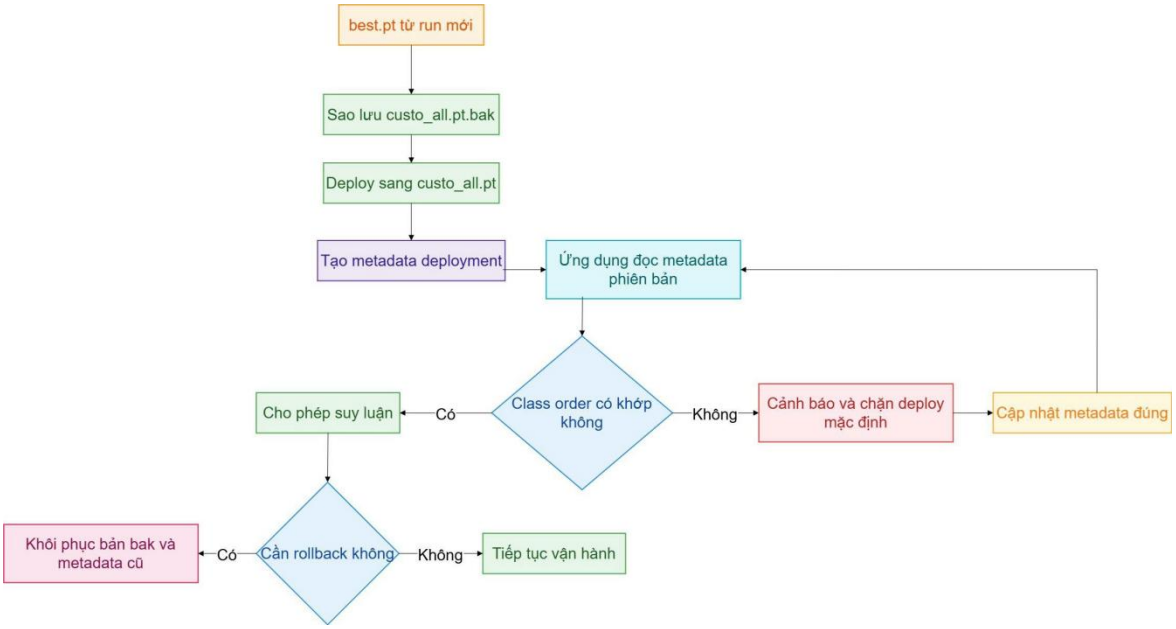
5.6 Kiểm soát phiên bản triển khai và cơ chế rollback

Sau mỗi lần huấn luyện, việc triển khai mô hình không chỉ là thay thế file trọng số mà cần đi kèm cơ chế kiểm soát phiên bản để bảo đảm an toàn vận hành. Trong hệ thống, bản mô hình mới chỉ được đưa vào chạy chính thức khi đã lưu bản dự phòng của phiên bản hiện tại và ghi đầy đủ metadata triển khai. Cấu trúc này giúp nhóm luôn biết chính xác mô hình nào đang hoạt động, mô hình nào là bản trước đó và điều kiện huấn luyện nào đã tạo ra phiên bản tương ứng.

Cơ chế rollback được thiết kế để xử lý nhanh các trường hợp phiên bản mới phát sinh lỗi hoặc suy giảm chất lượng trên bộ hard-case. Khi cần khôi phục, hệ thống có thể quay lại checkpoint ổn định trước đó cùng metadata tương ứng mà không cần huấn luyện lại từ đầu. Việc rollback theo cặp trọng số-metadata giúp tránh lỗi lệch cấu hình, đặc biệt khi thứ tự lớp hoặc tham số hậu xử lý đã thay đổi giữa các lần cập nhật.

Một thành phần kiểm soát quan trọng khác là đối chiếu class order giữa dataset và metadata deployment trước khi cho phép suy luận chính thức. Nếu phát hiện không khớp, hệ thống phát cảnh báo và yêu cầu đồng bộ lại phiên bản để ngăn sai lệch class id ở đầu ra. Cơ chế này giúp giảm rủi ro vận hành âm thầm, vì lỗi class mismatch thường khó phát hiện bằng quan sát bề mặt nhưng ảnh hưởng trực tiếp đến kết quả đếm.

Nhờ kiểm soát phiên bản và rollback có quy tắc, pipeline duy trì được tính ổn định trong quá trình cải tiến liên tục. Đây là điều kiện cần để hệ thống có thể mở rộng dữ liệu, cập nhật mô hình và tinh chỉnh rule mà vẫn giữ độ tin cậy khi triển khai thực tế.



Hình 5.1 Deploy metadata rollback

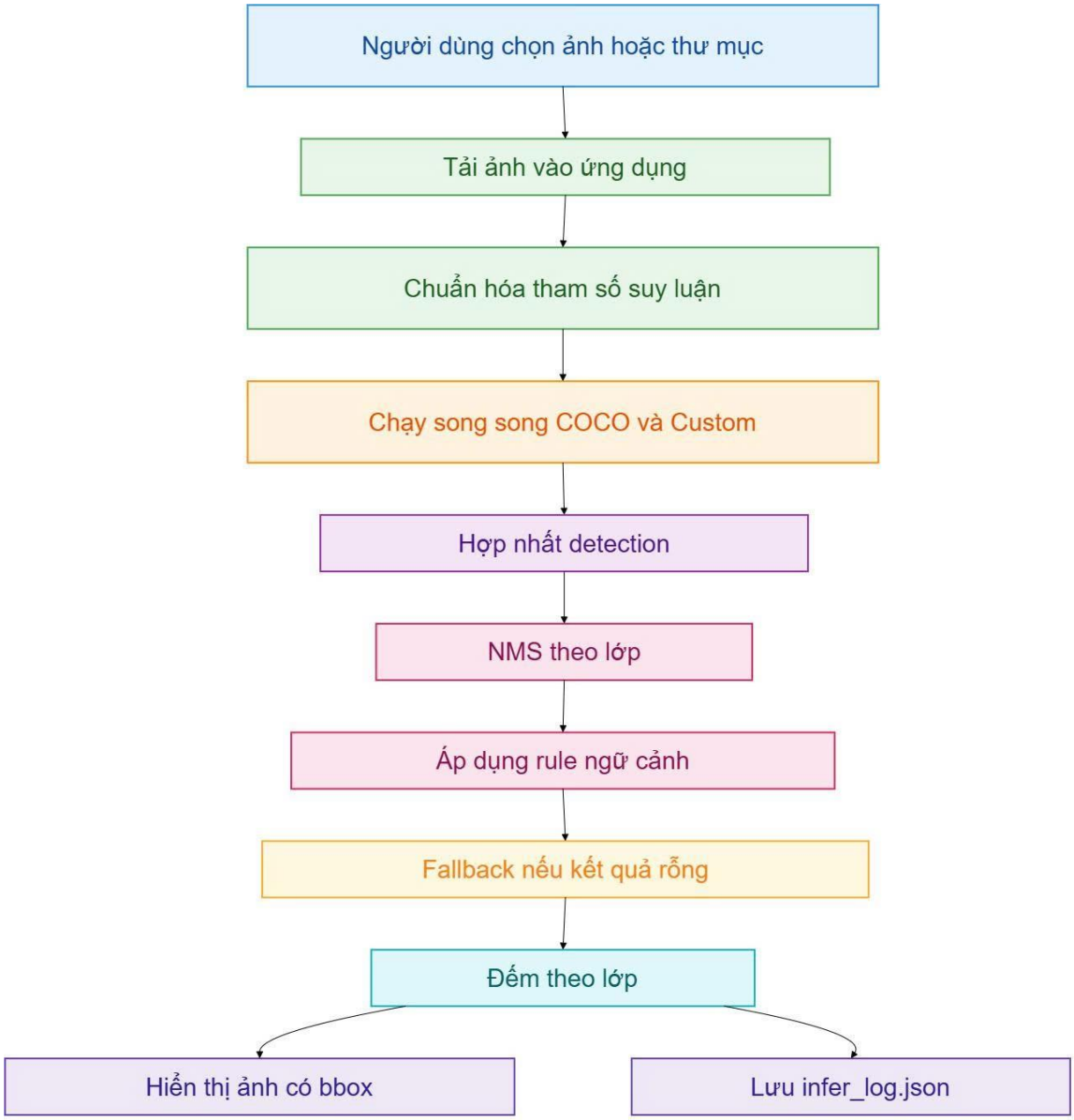
CHƯƠNG 6: PIPELINE SUY LUẬN VÀ HẬU XỬ LÝ

6.1 Luồng suy luận song song

Luồng suy luận của hệ thống được khởi tạo khi người dùng chọn một ảnh đơn hoặc một thư mục ảnh trên giao diện Streamlit. Ngay tại bước đầu, ảnh đầu vào được chuẩn hóa theo cấu hình inference hiện hành, bao gồm ngưỡng confidence, kích thước ảnh imgsz, giới hạn số lượng dự đoán max_det và ngưỡng IoU phục vụ lọc chồng lấp. Việc chuẩn hóa này giúp pipeline duy trì tính nhất quán giữa các lần chạy, hạn chế sai khác kết quả khi thay đổi điều kiện dữ liệu hoặc cấu hình kiểm thử.

Sau bước tiền xử lý, hệ thống thực hiện suy luận song song trên hai nguồn mô hình. Mô hình COCO pretrained đảm nhiệm vai trò phát hiện tổng quát các đối tượng phổ biến trong ảnh, còn mô hình custom custo_all tập trung nhận diện bốn lớp mục tiêu của đề tài gồm dumbbell, flower, fruit và tree. Mỗi nhánh suy luận trả về tập detection với thông tin bounding box, class id và độ tin cậy tương ứng. Cách tổ chức song song này giúp hệ thống tận dụng đồng thời tri thức tổng quát từ COCO và độ chuyên biệt từ mô hình custom.

Ý nghĩa chính của cơ chế song song nằm ở việc tăng độ bao phủ phát hiện trong bối cảnh ảnh thực tế phức tạp. Khi mô hình custom bỏ sót đối tượng do che khuất, kích thước nhỏ hoặc nền nhiễu, kết quả từ COCO có thể đóng vai trò bổ trợ; ngược lại, nhánh custom bảo đảm độ đúng lớp cho các đối tượng trọng tâm của bài toán. Vì vậy, luồng suy luận song song được xem là nền tảng của toàn bộ pipeline, tạo đầu vào đa nguồn để các bước hợp nhất và hậu xử lý phía sau tạo ra kết quả đếm ổn định, nhất quán và có giá trị sử dụng thực tế.



Hình 6.1 Sơ đồ luồng xử lý end-to-end

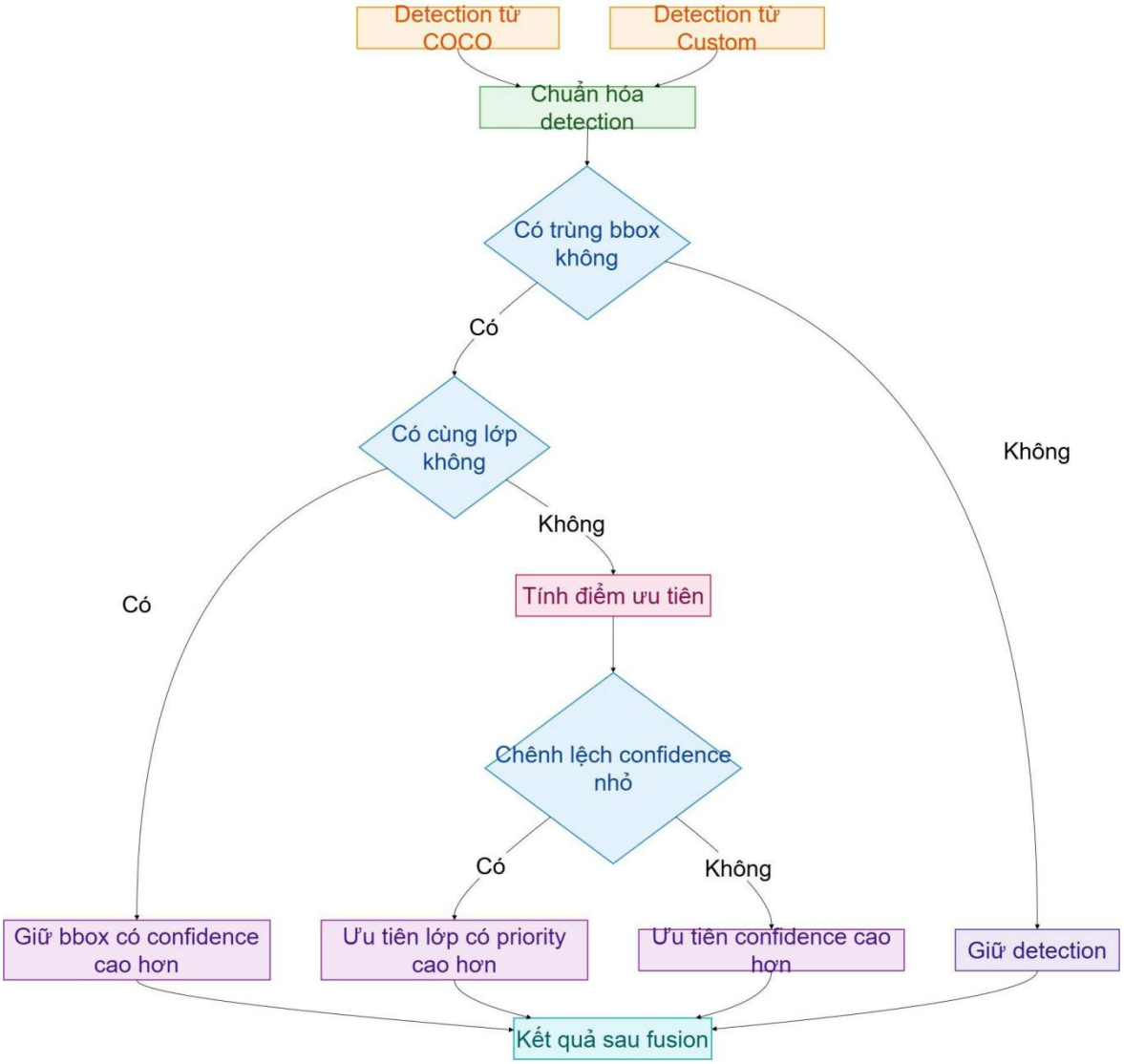
6.2 Hợp nhất detections và ưu tiên lớp

Sau khi hoàn tất suy luận song song, các kết quả từ mô hình COCO và mô hình custom được đưa về cùng một định dạng chuẩn để bảo đảm tính đồng nhất trước khi xử lý tiếp theo. Bước canonicalize này gồm chuẩn hóa tên lớp, cấu trúc bounding box, nguồn dự đoán và độ tin cậy của từng detection. Việc chuẩn hóa có ý nghĩa quan trọng vì hai mô hình có thể khác nhau về cách biểu diễn lớp hoặc định dạng dữ liệu đầu ra; nếu không đồng bộ ngay từ đầu, pipeline dễ phát sinh lỗi nhầm lớp, trùng box hoặc sai thống kê khi đếm theo lớp.

Trên tập detection đã chuẩn hóa, hệ thống áp dụng cơ chế ưu tiên theo nguyên tắc phục vụ mục tiêu bài toán. Khi một box từ custom trùng vùng với một box từ COCO, kết quả từ custom được ưu tiên giữ lại để bảo toàn bốn lớp mục tiêu của đề tài. Với các trường hợp nhiều box cùng lớp chồng lấp, hệ thống chỉ giữ box có

confidence cao nhất nhằm giảm nhiễu và hạn chế đếm thừa. Cách xử lý này giúp kết quả cuối gọn hơn, ổn định hơn và bám sát năng lực nhận diện chuyên biệt của mô hình chính.

Đối với các cặp lớp dễ nhầm lẫn như flower với fruit hoặc dumbbell với tree, pipeline sử dụng đồng thời CLASS_PRIORITY và SOURCE_PRIORITY để tính điểm ưu tiên tổng hợp cho từng detection. Nếu chênh lệch confidence không đủ lớn để kết luận rõ ràng, hệ thống sẽ ưu tiên lớp có mức priority cao hơn theo cấu hình đã thiết lập. Nhờ đó, bước hợp nhất không chỉ loại trùng mà còn xử lý xung đột nhãn một cách có kiểm soát. Kết quả sau hợp nhất vì vậy vừa giữ được các phát hiện đáng tin cậy từ COCO trong trường hợp custom suy yếu, vừa bảo đảm đầu ra cuối cùng phản ánh đúng lớp mục tiêu và hỗ trợ đếm chính xác hơn.



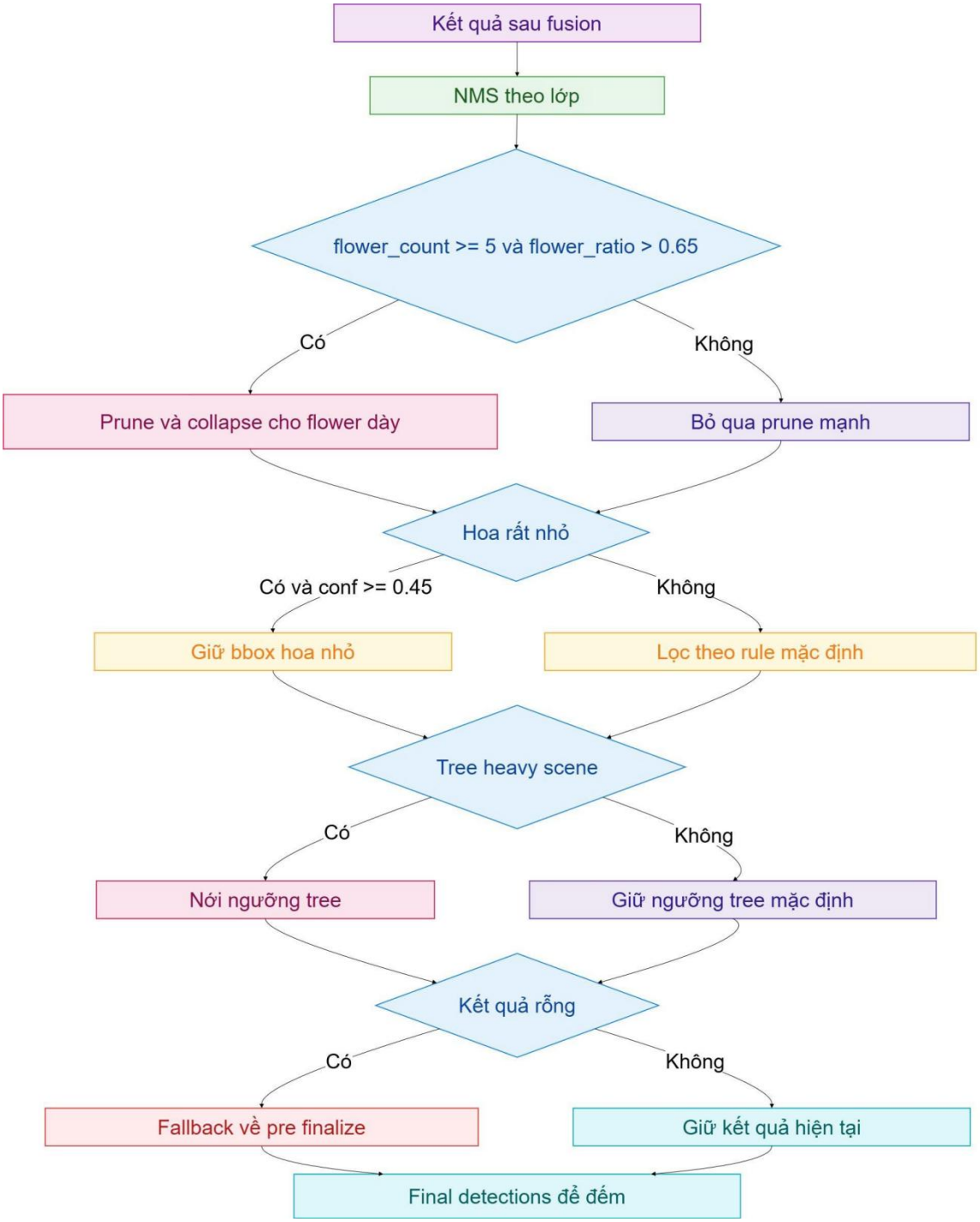
Hình 6.2 Sơ đồ hợp nhất detection và ưu tiên lớp

6.3 Rule hậu xử lý

Sau bước hợp nhất detection, hệ thống tiếp tục đi qua chuỗi hậu xử lý nhằm ổn định chất lượng đếm theo lớp. Các quy tắc chính được áp dụng gồm NMS theo lớp, ràng buộc một đối tượng tương ứng một bounding box, gom trùng lặp ở cảnh hoa dày, gom trùng lặp ở cảnh thưa, loại nhiễu hoa mật độ cao và giảm nhầm lẫn chéo giữa các lớp. Mục tiêu của giai đoạn này không phải tăng số lượng box, mà là tinh lọc đầu ra để giảm dự đoán dư và giữ lại các dự đoán thực sự có giá trị cho bước thống kê cuối.

Một nguyên tắc quan trọng của pipeline là các rule không kích hoạt đồng loạt, mà được bật theo điều kiện ngữ cảnh. Với lớp flower, các thao tác prune hoặc collapse chỉ chạy khi số lượng hoa đủ lớn và tỉ lệ hoa chiếm ưu thế trong ảnh, điển hình khi flower_count >= 5 và flower_ratio > 0.65. Cách giới hạn theo bối cảnh giúp tránh can thiệp quá mạnh trong ảnh thưa, từ đó giảm nguy cơ làm mất đối tượng thật. Bên cạnh đó, các hoa rất nhỏ vẫn được giữ nếu có độ tin cậy đạt ngưỡng, ví dụ area_ratio < 0.0003 nhưng confidence >= 0.45, nhằm hạn chế lỗi đếm thiếu ở nhóm tiny object.

Đối với lớp tree, hệ thống có cơ chế nhận diện bối cảnh tree-heavy khi số lượng cây trong ảnh vượt ngưỡng nhất định, chẳng hạn tree_n >= 3. Khi cờ ngữ cảnh này bật, ngưỡng giữ tree được nới xuống để tăng khả năng bảo toàn detection quan trọng, cụ thể giảm confidence tối thiểu và giảm điều kiện diện tích box. Đồng thời, nếu số lượng flower thấp, nhánh suppress weak tree được bỏ qua để tránh loại nhầm tree trong những cảnh không có mật độ hoa đáng kể. Cách điều phối rule theo từng tình huống giúp hệ thống cân bằng giữa giảm overcount và giảm undercount, đồng thời duy trì độ ổn định trên các ảnh khó có mật độ đối tượng biến thiên lớn.



Hình 6.3 Sơ đồ rule hậu xử lý theo ngữ cảnh

6.4 Đếm số lượng và log kết quả

Sau khi hoàn tất hậu xử lý, hệ thống thực hiện đếm số lượng đối tượng bằng cách tổng hợp số bounding box còn lại theo từng lớp mục tiêu. Kết quả đếm cho mỗi ảnh gồm số lượng dumbbell, flower, fruit và tree được hiển thị trực tiếp trên giao diện để người dùng theo dõi ngay sau khi suy luận. Đồng thời, ảnh đầu ra có gắn bounding box cũng được lưu lại nhằm phục vụ đối chiếu trực quan giữa kết quả thống kê và vị trí dự đoán thực tế trên ảnh.

Khi bật chế độ debug=True, hàm finalize_frame_detections_for_count ghi chi tiết trạng thái qua từng tầng xử lý dưới dạng stage_stats. Các mốc theo dõi thường bao gồm sau NMS, sau xử lý nhầm lớp chéo, sau lọc riêng cho flower, sau scene rules và kết quả cuối cùng. Ở mỗi mốc, hệ thống lưu thêm class_counts để thể hiện số lượng đối tượng theo lớp trước và sau khi áp dụng từng rule. Cách ghi vết theo từng tầng giúp nhóm nhìn rõ tác động của mỗi bước hậu xử lý thay vì chỉ quan sát một kết quả tổng cuối cùng.

Toàn bộ thông tin này được nối thêm vào infer_log.json trong thư mục tmp_test_outputs, tạo thành nguồn dữ liệu chuẩn cho phân tích định lượng. Từ log, nhóm có thể tính MAE trên bộ hard-case, so sánh mức cải thiện hoặc suy giảm giữa các phiên bản, đồng thời xác định chính xác stage nào gây giảm detection mạnh nhất. Nhờ đó, việc tinh chỉnh ở các vòng sau có cơ sở rõ ràng, tránh điều chỉnh cảm tính và giúp pipeline tiến hóa theo hướng ổn định, có kiểm chứng.

6.5 Khả năng fallback và sanity check

Khi đi qua toàn bộ chuỗi hậu xử lý, hệ thống có thể gặp tình huống đặc biệt là toàn bộ detection bị loại bỏ do rule quá chặt. Để tránh kết quả rỗng không phản ánh đúng nội dung ảnh, pipeline kích hoạt cơ chế fallback: nếu đầu ra cuối cùng không còn box nào thì hệ thống trả lại snapshot pre-finalize, với điều kiện snapshot này có ít nhất một lớp mục tiêu từ mô hình custom như dumbbell, flower, fruit hoặc tree. Cách xử lý này giúp bảo toàn thông tin phát hiện hợp lệ trong các ca khó và giảm rủi ro xuất hiện kết quả "đếm bằng 0" không mong muốn.

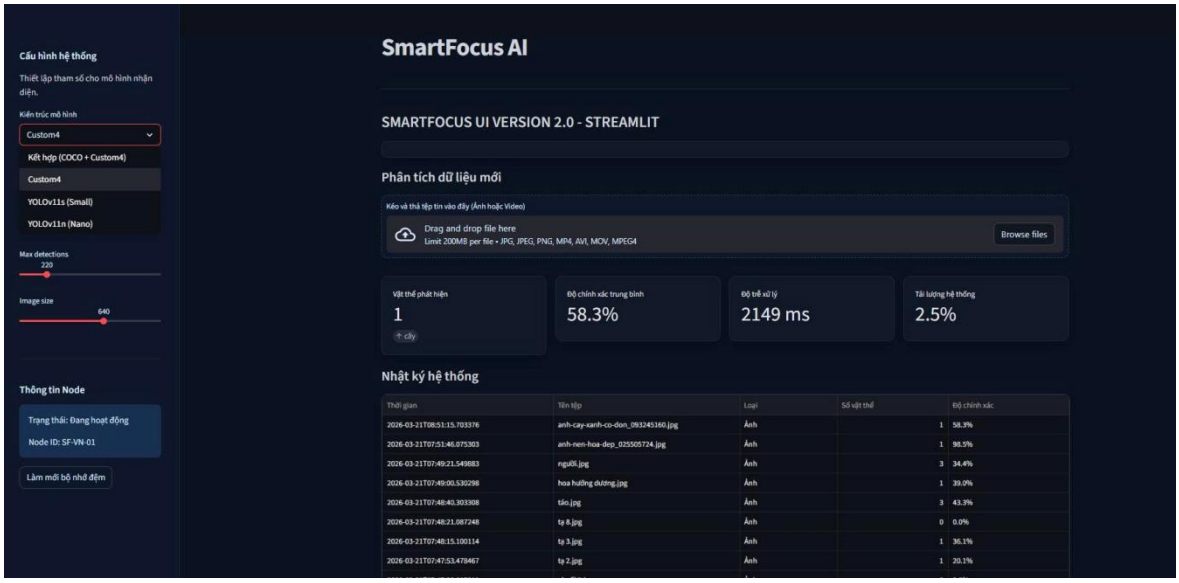
Song song với fallback trong giai đoạn suy luận, hệ thống còn duy trì lớp kiểm tra sanity trước khi vận hành mô hình. Hàm check_class_order_and_warn() được dùng để đối chiếu thứ tự lớp giữa dataset hiện tại và metadata deployment. Nếu phát hiện sai khác, ứng dụng sẽ phát cảnh báo và yêu cầu cập nhật metadata trước khi tiếp tục inference. Cơ chế này đặc biệt quan trọng khi dữ liệu vừa được mở rộng, đổi alias hoặc tinh chỉnh nhãn, vì chỉ một sai lệch nhỏ ở class order cũng có thể gây nhầm class id và dẫn đến thống kê sai toàn bộ.

Việc kết hợp fallback và sanity check tạo thành lớp bảo vệ hai chiều cho pipeline. Fallback giúp hệ thống không mất toàn bộ kết quả trong các trường hợp rule hậu xử lý tác động quá mạnh, còn sanity check bảo đảm đầu ra luôn nhất quán với cấu trúc lớp đã định nghĩa từ khâu huấn luyện đến triển khai. Nhờ đó, hệ thống vận hành ổn định hơn trong thực tế, giảm lỗi logic khó phát hiện và nâng độ tin cậy cho kết quả đếm cuối cùng.

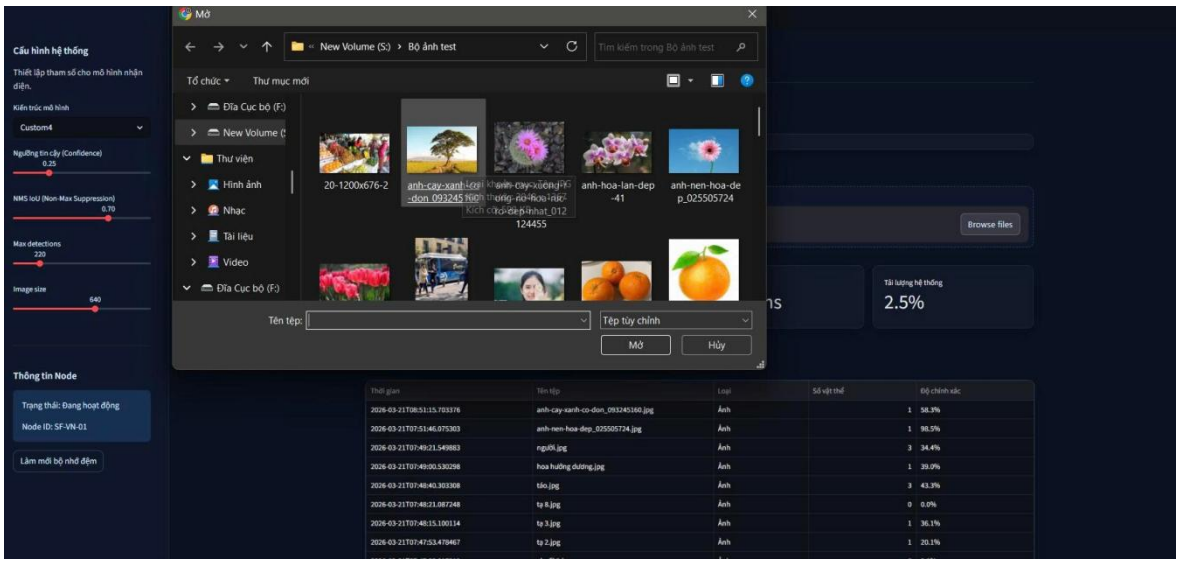
6.6 Giao diện hệ thống và minh họa kết quả

Khi thực hiện lệnh npm start, hệ thống sẽ khởi động ứng dụng và tự động mở giao diện Streamlit trên trình duyệt. Tại đây, người dùng có thể bắt đầu quá trình quét và phân tích ảnh thông qua việc tải lên dữ liệu đầu vào.

Giao diện Streamlit cung cấp khu vực upload ảnh hoặc video, cho phép người dùng lựa chọn file từ máy hoặc kéo thả trực tiếp. Sau khi tải lên, hệ thống sẽ sử dụng mô hình đã được cấu hình để tiến hành nhận diện đối tượng và hiển thị kết quả ngay trên giao diện, bao gồm số lượng đối tượng phát hiện, độ chính xác và các thông số liên quan. Điều này giúp người dùng dễ dàng tương tác và kiểm tra hiệu suất của hệ thống một cách trực quan.



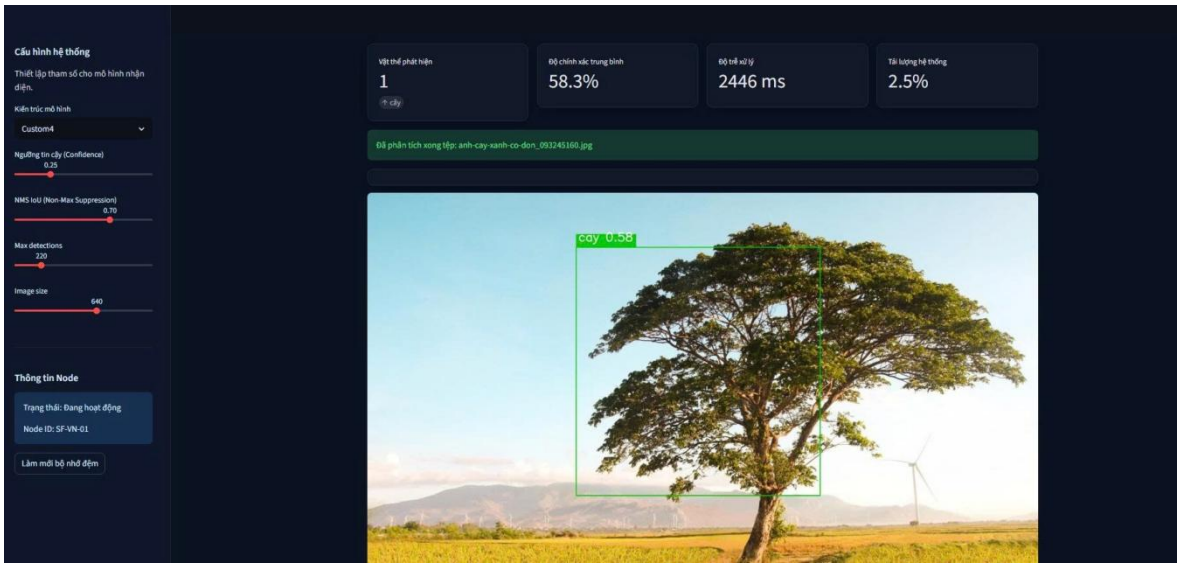
Hình 6.4 Giao diện và tùy chọn model



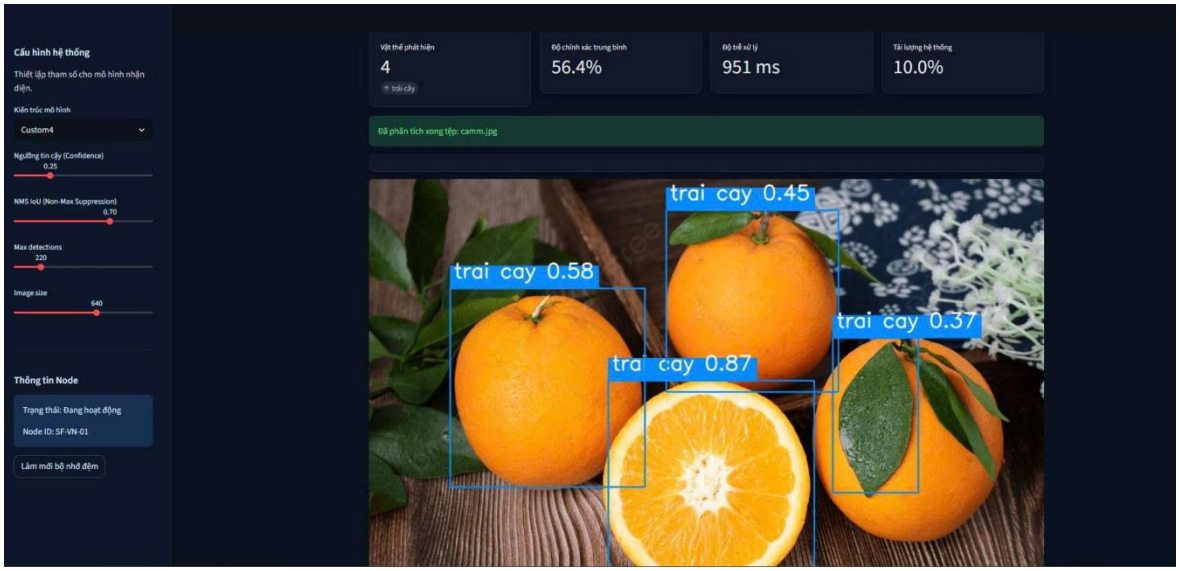
Hình 6.5 Quy trình chọn và tải ảnh lên

Ảnh đầu vào đã được hệ thống quét và nhận diện thành công bằng mô hình Custom do người dùng huấn luyện. Kết quả hiển thị cho thấy hệ thống đã phát hiện chính xác đối tượng "cây, trái cây, hoa" trong ảnh và khoanh vùng bằng bounding box kèm theo nhãn (label). Điều này chứng tỏ mô hình đã học tốt đặc trưng của đối tượng và có khả năng áp dụng vào thực tế.

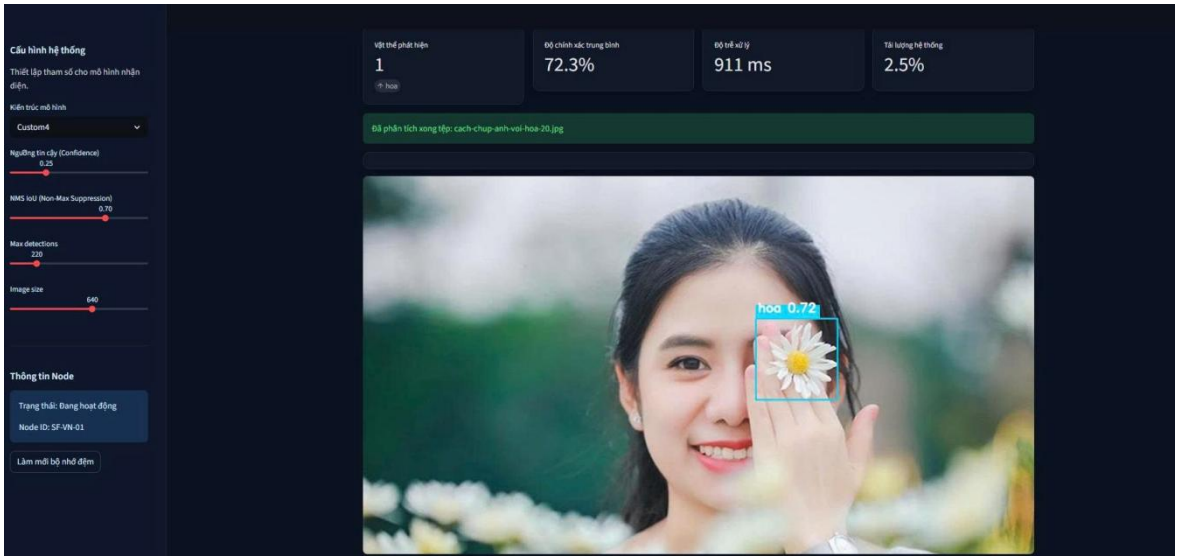
Bên cạnh đó, các thông số như số lượng đối tượng phát hiện, độ chính xác trung bình và thời gian xử lý cũng được hiển thị rõ ràng trên giao diện, giúp đánh giá hiệu suất của mô hình. Kết quả này cho thấy hệ thống hoạt động ổn định và mô hình custom có khả năng nhận diện đúng đối tượng trong dữ liệu thực tế.



Hình 6.6 Kết quả quét đúng 1



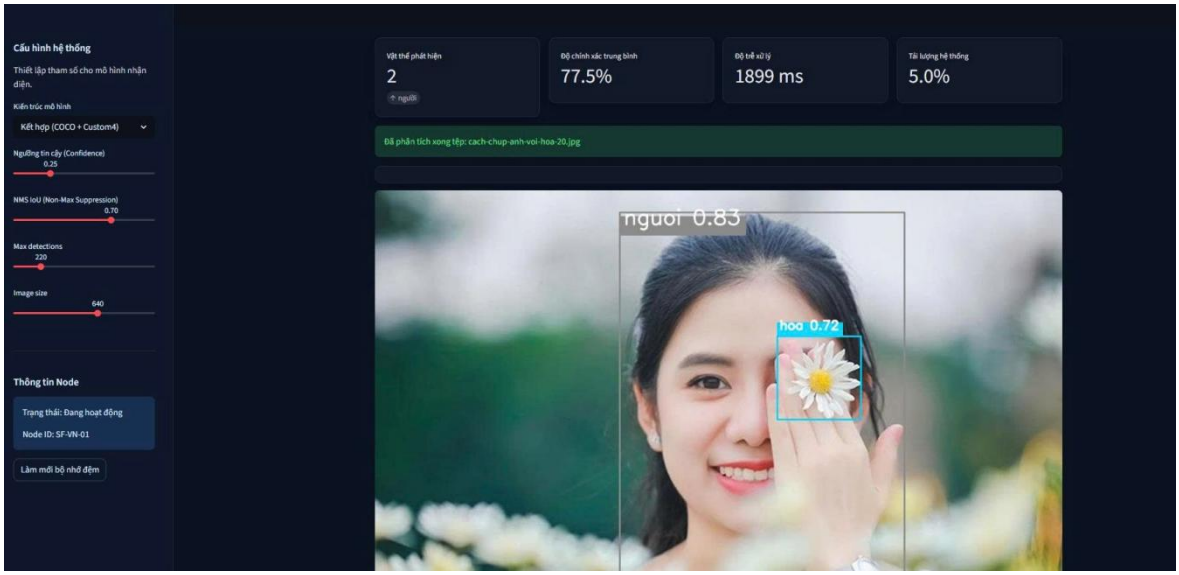
Hình 6.7 Kết quả quét đúng 2



Hình 6.8 Kết quả quét đúng 3

Ảnh đầu vào đã được hệ thống quét và nhận diện thành công khi sử dụng chế độ kết hợp giữa mô hình Custom và COCO. Kết quả cho thấy hệ thống phát hiện đồng thời nhiều đối tượng trong ảnh, bao gồm "người" (từ bộ dữ liệu COCO) và "hoa" (từ mô hình Custom đã huấn luyện), mỗi đối tượng đều được khoanh vùng bằng bounding box và gắn nhãn với độ tin cậy tương ứng.

Việc kết hợp hai mô hình giúp tận dụng ưu điểm của cả hai: mô hình COCO nhận diện tốt các đối tượng phổ biến, trong khi mô hình Custom tập trung vào các đối tượng đặc thù do người dùng huấn luyện. Nhờ đó, hệ thống có khả năng nhận diện đa dạng và chính xác hơn trong cùng một ảnh. Các thông số như số lượng đối tượng phát hiện, độ chính xác trung bình và thời gian xử lý cũng được hiển thị, cho thấy hiệu suất hoạt động ổn định và hiệu quả của phương pháp kết hợp mô hình.

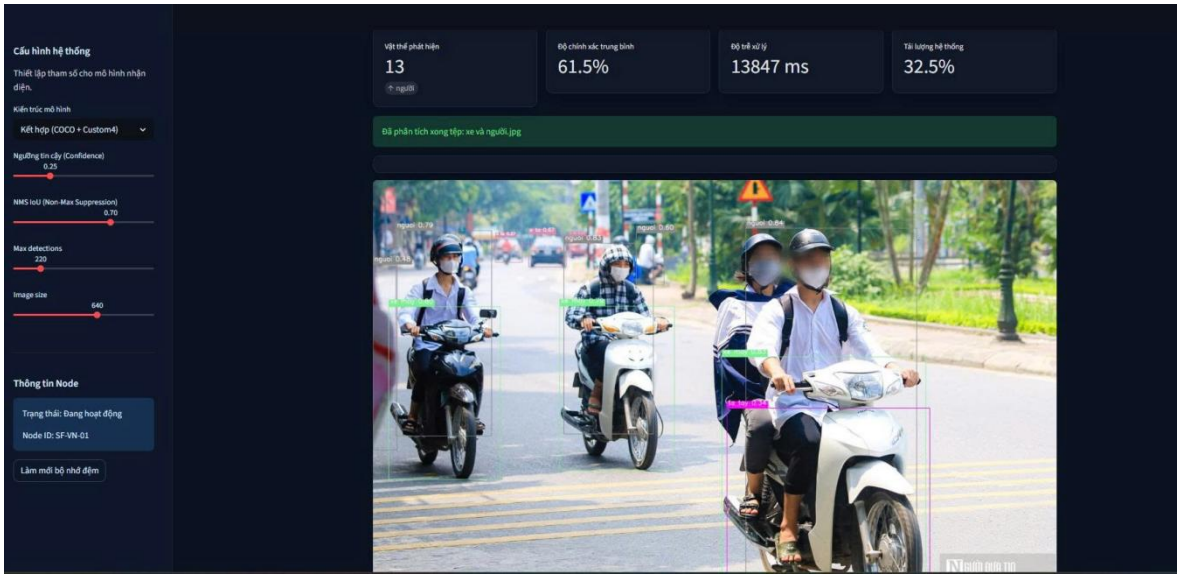


Hình 6.9 Kết quả quét đúng kết hợp hai model

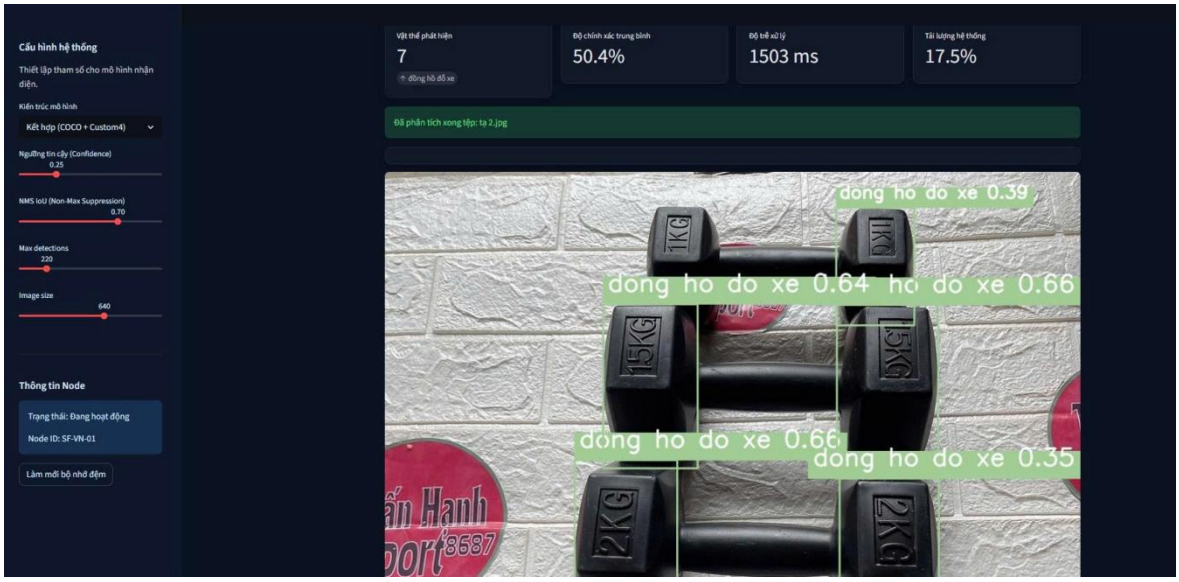
Khi sử dụng chế độ kết hợp giữa mô hình Custom và COCO, hệ thống vẫn xuất hiện hiện tượng nhiễu bounding box và độ chính xác chưa ổn định trong một số trường hợp. Cụ thể, ở các ảnh có nhiều đối tượng như giao thông, mô hình phát hiện được nhiều vật thể nhưng đồng thời cũng sinh ra các khung nhận diện dư thừa hoặc chồng lấn, làm giảm độ chính xác tổng thể. Tỷ lệ lỗi hệ thống trong trường hợp này tăng lên đáng kể, cho thấy sự ảnh hưởng của nhiễu trong quá trình dự đoán.

Đối với lớp đối tượng tạ (dumbbell) trong mô hình Custom, kết quả nhận diện chưa thực sự hiệu quả. Một số đối tượng được phát hiện với độ tin cậy thấp hoặc bị bỏ sót, nguyên nhân có thể do dữ liệu huấn luyện chưa đủ đa dạng hoặc số lượng mẫu còn hạn chế. Ngoài ra, sự khác biệt về góc nhìn, ánh sáng và nền ảnh cũng ảnh hưởng đến khả năng học đặc trưng của mô hình.

Tổng thể, kết quả cho thấy việc kết hợp hai mô hình giúp tăng khả năng nhận diện đa đối tượng, tuy nhiên vẫn cần cải thiện bằng cách tối ưu lại dữ liệu huấn luyện, tăng số lượng mẫu cho lớp Custom và điều chỉnh các tham số như confidence threshold hoặc NMS để giảm nhiễu và nâng cao độ chính xác.

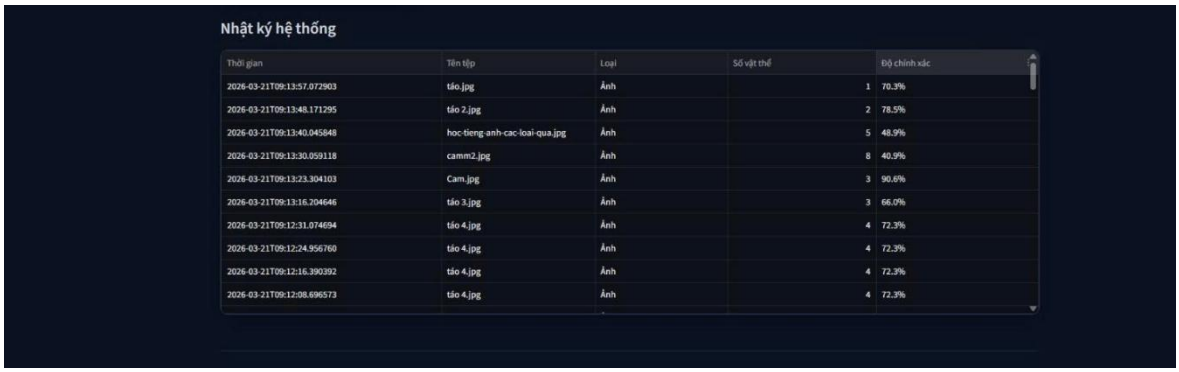


Hình 6.10 Kết quả quét sai 1



Hình 6.11 Kết quả quét sai 2

Nhật ký hệ thống là thành phần dùng để ghi lại toàn bộ quá trình xử lý dữ liệu của hệ thống SmartFocus AI. Mỗi lần người dùng tải ảnh hoặc video lên để phân tích, hệ thống sẽ tự động lưu lại các thông tin liên quan nhằm phục vụ cho việc theo dõi và đánh giá.



Nhật ký hệ thống

Thời gian	Tên tệp	Loại	Số vật thể	Độ chính xác
2026-03-21T09:13:57.072903	táo.jpg	Ảnh	1	70.3%
2026-03-21T09:13:48.171295	táo 2.jpg	Ảnh	2	78.5%
2026-03-21T09:13:40.045848	hoc-tieng-anh-cac-loai-qua.jpg	Ảnh	5	48.9%
2026-03-21T09:13:30.059118	camm2.jpg	Ảnh	8	40.9%
2026-03-21T09:13:23.304103	Cam.jpg	Ảnh	3	90.6%
2026-03-21T09:13:16.204646	táo 3.jpg	Ảnh	3	66.0%
2026-03-21T09:12:31.074694	táo 4.jpg	Ảnh	4	72.3%
2026-03-21T09:12:24.956760	táo 4.jpg	Ảnh	4	72.3%
2026-03-21T09:12:16.390392	táo 4.jpg	Ảnh	4	72.3%
2026-03-21T09:12:08.696573	táo 4.jpg	Ảnh	4	72.3%

Hình 6.12 Nhật ký hệ thống

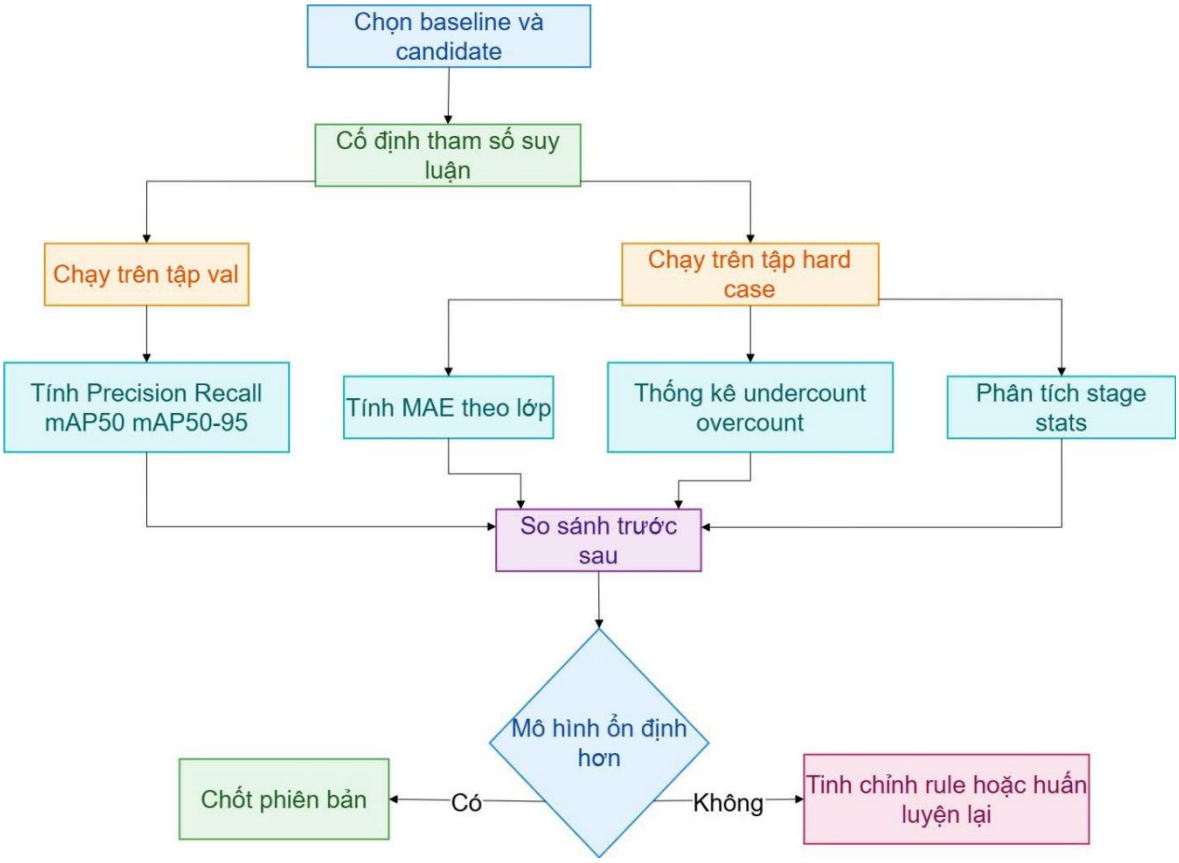
CHƯƠNG 7 – THỬ NGHIỆM KẾT QUẢ VÀ KẾT LUẬN

7.1 Thiết lập thử nghiệm

Thử nghiệm của đề tài được tổ chức theo hai mức nhằm phản ánh đầy đủ hiệu quả vận hành của hệ thống. Mức thứ nhất là đánh giá tổng thể trên tập validation chuẩn tại model_custom/dataset/val, dùng để quan sát chất lượng mô hình trong điều kiện dữ liệu gần với phân bố huấn luyện. Mức thứ hai là đánh giá chuyên sâu trên tập hard-case, gồm các ảnh từng xuất hiện lỗi đếm lớn trong quá trình kiểm thử, được lưu trong tmp_test_outputs và các tệp log như hoa_test_log.json. Cách chia này giúp tách rõ năng lực tổng quát của mô hình và độ bền của pipeline trong các tình huống khó.

Để bảo đảm so sánh công bằng giữa các phiên bản, toàn bộ tham số suy luận được giữ cố định trong từng đợt benchmark, bao gồm confidence threshold, kích thước ảnh đầu vào imgsz, ngưỡng IoU của NMS và max_det. Mỗi lần thay đổi mô hình hoặc rule hậu xử lý đều được chạy lại trên cùng bộ ảnh và cùng cấu hình, từ đó hạn chế sai lệch do điều kiện chạy khác nhau. Quy trình này cho phép đối chiếu trực tiếp giữa baseline và candidate theo cả detection lẫn counting.

Trong các phiên chạy phân tích lỗi, chế độ debug được bật để ghi stage_stats qua từng bước hậu xử lý. Dữ liệu này cho biết số lượng box bị giảm ở từng stage, giúp xác định chính xác rule nào gây mất detection nhiều nhất. Nhờ vậy, việc tinh chỉnh không dừng ở mức quan sát kết quả cuối, mà có thể bám theo nguyên nhân tại từng khâu xử lý trong pipeline.



Hình 7.1 Sơ đồ quy trình đánh giá

7.2 Kết quả đánh giá tổng thể

Đánh giá tổng thể sử dụng bốn chỉ số chuẩn của bài toán detection gồm Precision, Recall, mAP50 và mAP50-95. Ở các lần huấn luyện gần nhất, mô hình chính đạt Precision xấp xỉ 0.71, Recall xấp xỉ 0.39, mAP50 xấp xỉ 0.41 và mAP50-95 xấp xỉ 0.26 trên tập validation. Các giá trị này cho thấy hệ thống đã đạt mức phát hiện ổn định trong phạm vi dữ liệu hiện có, đồng thời giữ được độ tin cậy tương đối tốt ở đầu ra dự đoán.

Về ý nghĩa thực nghiệm, Precision ở mức cao hơn Recall cho thấy mô hình có xu hướng dự đoán thận trọng, giảm nhiễu tốt nhưng vẫn còn dư địa cải thiện khả năng bao phủ đối tượng khó. Tuy nhiên, khi xét trong bối cảnh hệ thống có thêm tầng fusion và hậu xử lý phục vụ bài toán đếm, bộ chỉ số hiện tại vẫn đáp ứng yêu cầu triển khai của đồ án. So với baseline trước đó, kết quả mới thể hiện xu hướng cải thiện rõ ở các lớp khó như tree và dumbbell, phù hợp với tác động từ việc mở rộng dữ liệu và fine-tune theo lớp.

Tổng hợp lại, kết quả benchmark xác nhận hướng tối ưu của đề tài là hợp lý: nâng chất lượng mô hình nền bằng dữ liệu có kiểm soát, sau đó ổn định đầu ra bằng pipeline hậu xử lý theo ngữ cảnh. Đây là cơ sở để tiếp tục đánh giá sâu theo từng lớp và theo sai số đếm trong các mục tiếp theo.

7.3 Kết quả đếm theo lớp

Ngoài các chỉ số detection như mAP, đề tài tập trung đánh giá sai số đếm theo từng lớp thông qua MAE vì mục tiêu cuối cùng của hệ thống là thống kê số lượng vật thể. Kết quả thực nghiệm cho thấy lớp dumbbell và tree thường đạt MAE thấp hơn do đối tượng có kích thước tương đối lớn, hình dạng ổn định và mức chồng lắp không cao. Với lớp fruit, MAE ở mức trung bình vì kết quả đếm phụ thuộc nhiều vào mật độ xuất hiện trong ảnh và khả năng phân biệt với các vùng hoa hoặc lá có đặc trưng thị giác tương tự. Lớp flower tiếp tục là lớp khó nhất, có MAE cao nhất trong phần lớn thí nghiệm do kích thước nhỏ, phân bố dày và dễ phát sinh cả đếm thiếu lẫn đếm thừa.

Trên bộ hard-case, khi kích hoạt các rule chuyên biệt cho flower và giới hạn prune trong bối cảnh thưa, MAE có xu hướng giảm nhẹ nhưng ổn định hơn qua các lần chạy, đồng thời giảm số ca regression so với cách lọc cứng trước đó. Kết quả này cho thấy định hướng xử lý theo ngữ cảnh phù hợp hơn so với việc áp cùng một mức lọc cho mọi ảnh. Đối với mô hình specialist flower, các checkpoint giai đoạn đầu chưa vượt baseline một cách rõ rệt, vì vậy việc kết luận hiệu quả cần dựa trên phiên bản huấn luyện đầy đủ và benchmark lại trên cùng bộ ảnh chuẩn trước khi đưa vào pipeline chính thức.

7.4 Phân tích lỗi và cải tiến

Phân tích log theo stage cho thấy mức giảm detection lớn thường tập trung ở bước after_flower_pruning, qua đó chỉ ra nguyên nhân sai số không nằm hoàn toàn ở mô hình thô mà chịu tác động đáng kể từ tầng hậu xử lý. Một số thử nghiệm cũng xác nhận việc nới dense_conf_floor quá mạnh có thể gây regression, vì làm thay đổi cân bằng giữa giữ đối tượng thật và loại nhiễu. Do đó, chiến lược cải tiến được lựa chọn là điều chỉnh từng rule độc lập với biên độ nhỏ, theo dõi tác động sau mỗi vòng thay đổi thay vì nới ngưỡng toàn cục.

Các cải tiến đã áp dụng gồm mở điều kiện kích hoạt rule theo flower_count, giữ lại hoa nhỏ khi đạt mức confidence tối thiểu, và nới ngưỡng cho tree trong bối cảnh tree-heavy. Tổ hợp thay đổi này giúp giảm undercount ở các ảnh khó, trong khi overcount chỉ tăng nhẹ hoặc giữ ổn định ở mức chấp nhận được. Quan trọng hơn, hệ thống thể hiện xu hướng ổn định tốt hơn giữa các lần chạy trên cùng bộ kiểm thử.

Trong giai đoạn tối ưu, mỗi lần chỉnh sửa rule đều được kiểm tra lại trên bộ hard-20 để so sánh MAE cùng số lượng ảnh improved và worsened. Kết quả tổng hợp cho thấy kiểm soát theo ngữ cảnh là yếu tố quyết định trong việc nâng chất lượng đếm thực tế. Cách tiếp cận này giúp pipeline duy trì hiệu quả không chỉ ở

một tập ảnh cụ thể mà còn bền hơn khi triển khai trên dữ liệu vận hành có độ biến thiên cao.

7.5 Kết luận và hướng phát triển

Hệ thống đã xây dựng đầy đủ một pipeline phát hiện và đếm vật thể theo hướng triển khai thực tế, kết hợp suy luận song song giữa mô hình COCO và mô hình custom, hậu xử lý theo ngữ cảnh và cơ chế ghi log để phân tích lỗi theo từng stage. Kết quả thực nghiệm cho thấy phiên bản hiện tại cải thiện rõ so với baseline ở nhiều tình huống, đặc biệt với các lớp tree và dumbbell khi mở rộng dữ liệu và tinh chỉnh rule theo bối cảnh. Đối với lớp flower, bài toán vẫn còn khó do đặc trưng đối tượng nhỏ và mật độ cao, tuy nhiên hệ thống đã hình thành được hướng xử lý khả thi thông qua nhóm rule chuyên biệt và nhánh specialist hỗ trợ.

Trong giai đoạn tiếp theo, ưu tiên chính là hoàn thiện mô hình specialist cho flower bằng cách huấn luyện đủ epoch, sau đó benchmark lại trên cùng bộ val và hard-case để đánh giá công bằng với baseline hiện tại. Song song với đó, việc tiếp tục mở rộng dữ liệu theo các cảnh khó sẽ giúp giảm bias và tăng độ bao phủ cho mô hình. Ở tầng hậu xử lý, cần nghiên cứu thêm các cơ chế thích nghi thông minh hơn để giảm sai số cho flower nhưng vẫn kiểm soát được regression trên các lớp còn lại. Với nền tảng hiện có, pipeline đã đạt mức ổn định cần thiết để vận hành, đồng thời có khả năng mở rộng tốt khi bổ sung dữ liệu hoặc phát sinh yêu cầu mới.

TÀI LIỆU THAM KHẢO

Mankodiya, H., Jadav, D., Gupta, R., Tanwar, S., Hong, W.-C., & Sharma, R. (2022). OD-XAI: Explainable AI-based semantic object detection for autonomous vehicles. *Applied Sciences, 12*(11), 5310. https://doi.org/10.3390/app12115310

Nawaz, S. A., Li, J., Bhatti, U. A., Shoukat, M. U., & Ahmad, R. M. (2022). AI-based object detection: Latest trends in remote sensing, multimedia, and agriculture applications. *Frontiers in Plant Science, 13*, 1041514. https://doi.org/10.3389/fpls.2022.1041514

Matsuzaka, Y., & Yashiro, R. (2023). AI-based computer vision techniques and expert systems. *AI, 4*(1), 289–302. https://doi.org/10.3390/ai4010013

Silva, D. A., Smagulova, K., Elsheikh, A., Fouda, M. E., & Eltawil, A. M. (2025). A recurrent YOLOv8-based framework for event-based object detection. *Frontiers in Neuroscience, 18*, 1477979. https://doi.org/10.3389/fnins.2024.1477979

Terven, J., Córdova-Esparza, D.-M., & Romero-González, J.-A. (2023). A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction, 5*(4), 1680–1716. https://doi.org/10.3390/make5040083

Ahmad, T., Ma, Y., Yahya, M., Ahmad, B., Nazir, S., Haq, A. U., & Ali, R. (2020). Object detection through modified YOLO neural network. *Scientific Programming, 2020*, Article 8403262. https://doi.org/10.1155/2020/8403262